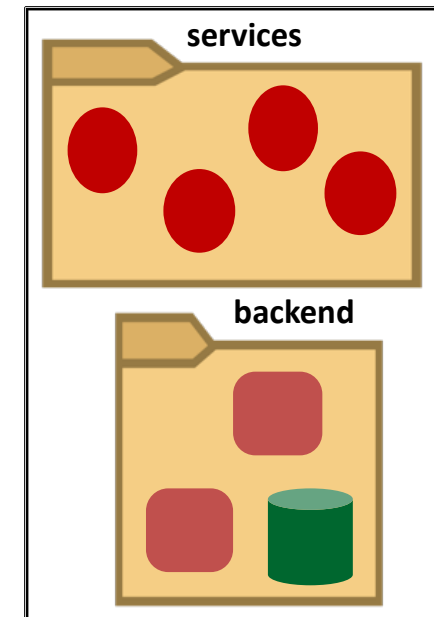
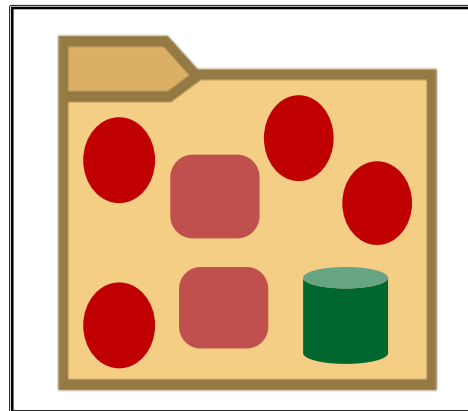


Microservicios

Diseño de Sistemas de Software

SOA y el Diseño monolítico

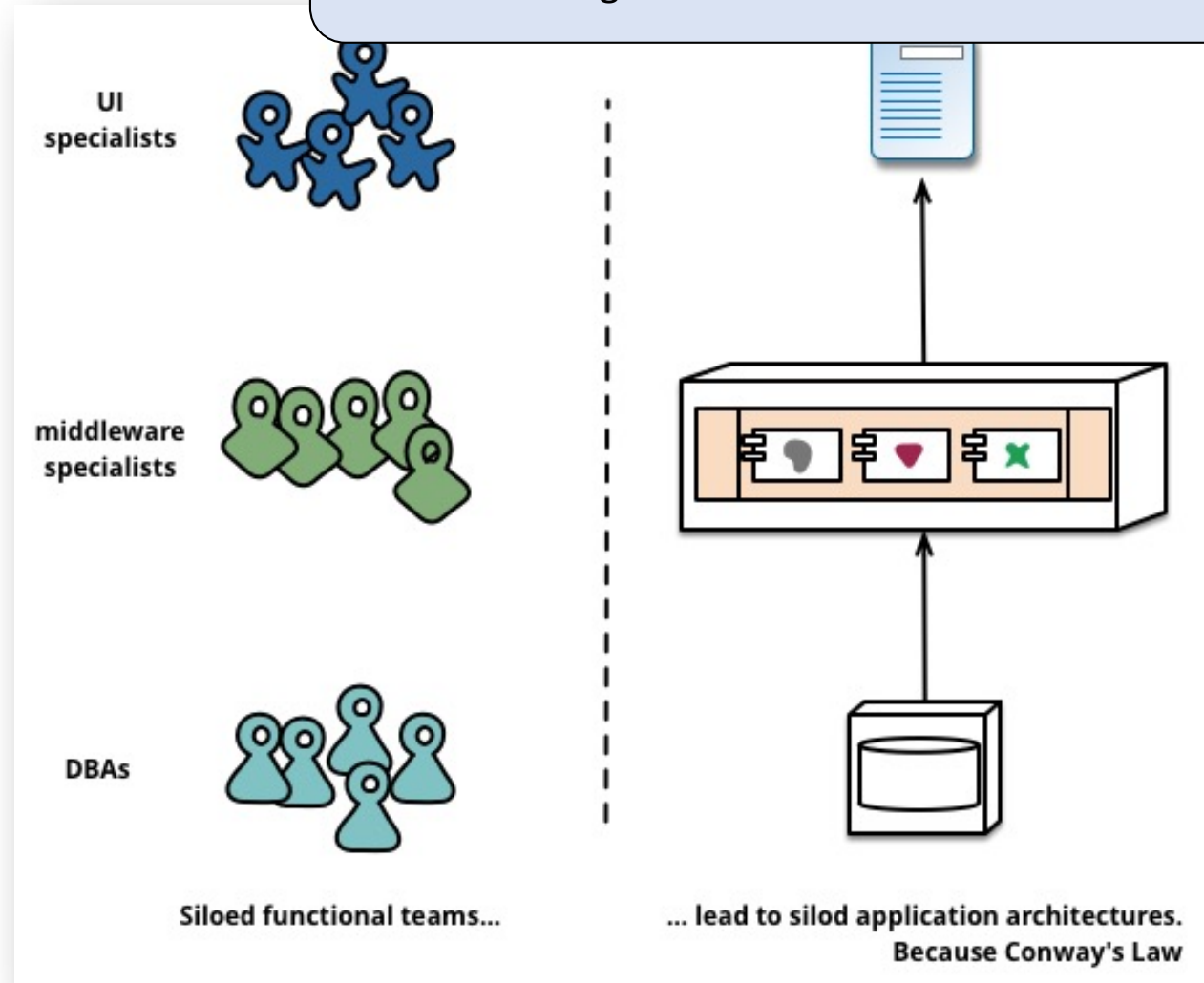
- Si bien las arquitecturas orientadas a servicios (SOA) son un estilo muy popular para las aplicaciones Web ...
- Todos los servicios (o lógica de negocio) de la aplicación están empaquetados en un **único artefacto para despliegue**
→ **el monolito**
 - Esto implica que cualquier cambio en la aplicación requiere un re-despliegue de toda la aplicación
 - Adicionalmente, si se desea escalar, debe escalarse toda la aplicación



Implicancias para el Desarrollo

- Los distintos equipos de desarrollo y operaciones trabajan sobre el mismo repositorio y sobre distintas partes de toda la aplicación, y no en forma **desacoplada**

Ley de Conway: An organization's application structure is going to be a close mimic of their IT organizational structure.



¿Porqué Microservicios?

- La forma en que un sistema de software **se empaqueta y se despliega** afecta directamente ciertos **atributos de calidad**
 - Escalabilidad, disponibilidad, mantenibilidad, performance, etc.
- Esto suele alinear bien con procesos/equipos ágiles
- En 2002, Amazon formuló un conjunto de reglas para sus desarrolladores, que luego derivó en el concepto de microservicios
- Muchas de las guías de diseño para SOA también aplican a microservicios
 - Entonces cuál es la diferencia?



En más detalle

- Un **(micro)servicio** debe ser lo suficientemente “pequeño” como para ser desarrollado por un único equipo y ser fácilmente testeable
 - **Principio de Responsabilidad Unica (POO)**, en el sentido que una clase o conjunto de clases deben tener una funcionalidad cohesiva y bajo acoplamiento con otras clases (que implementan otras responsabilidades)
 - El servicio debe “encapsular” cambios relacionados a su responsabilidad
- Un microservicio no solo involucra su código fuente (por ej., SpringBoot) sino la **capacidad de ser desplegado independientemente** de otros microservicios → runtime



Algunas definiciones

“Microservices are **small, autonomous services** that work together.” – Newman [Newman 2015]

“[...] the **microservice architectural style** is an approach to developing a single application as a **suite of small services**, each **running in its own process** [...]. These services are **built around business capabilities** and **independently deployable** by fully automated deployment machinery. There is a **bare minimum of centralized management** of these services, which may be written in different programming languages and use different data storage technologies.” – Lewis and Fowler [2015]

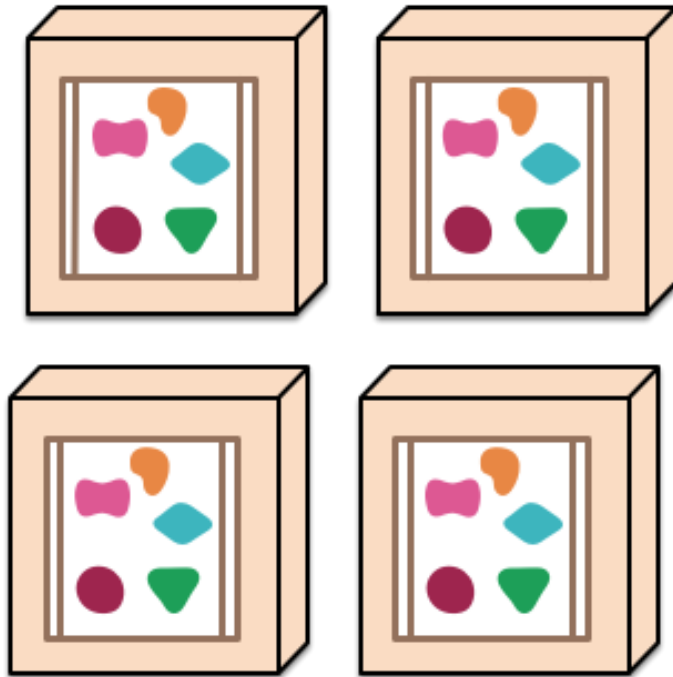


La idea (Fowler, 2014)

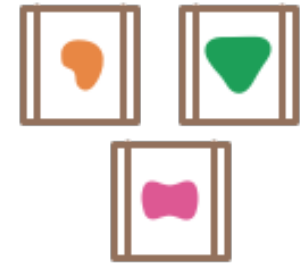
A monolithic application puts all its functionality into a single process...



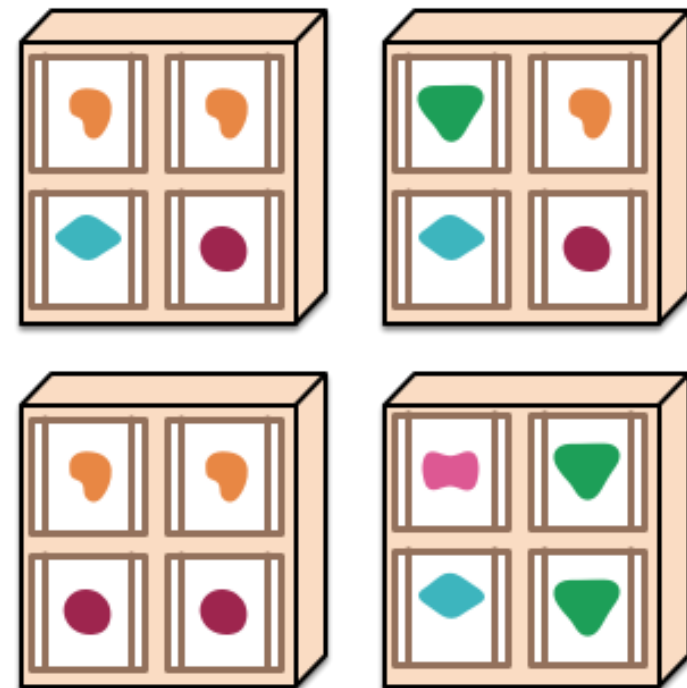
... and scales by replicating the monolith on multiple servers



A microservices architecture puts each element of functionality into a separate service...

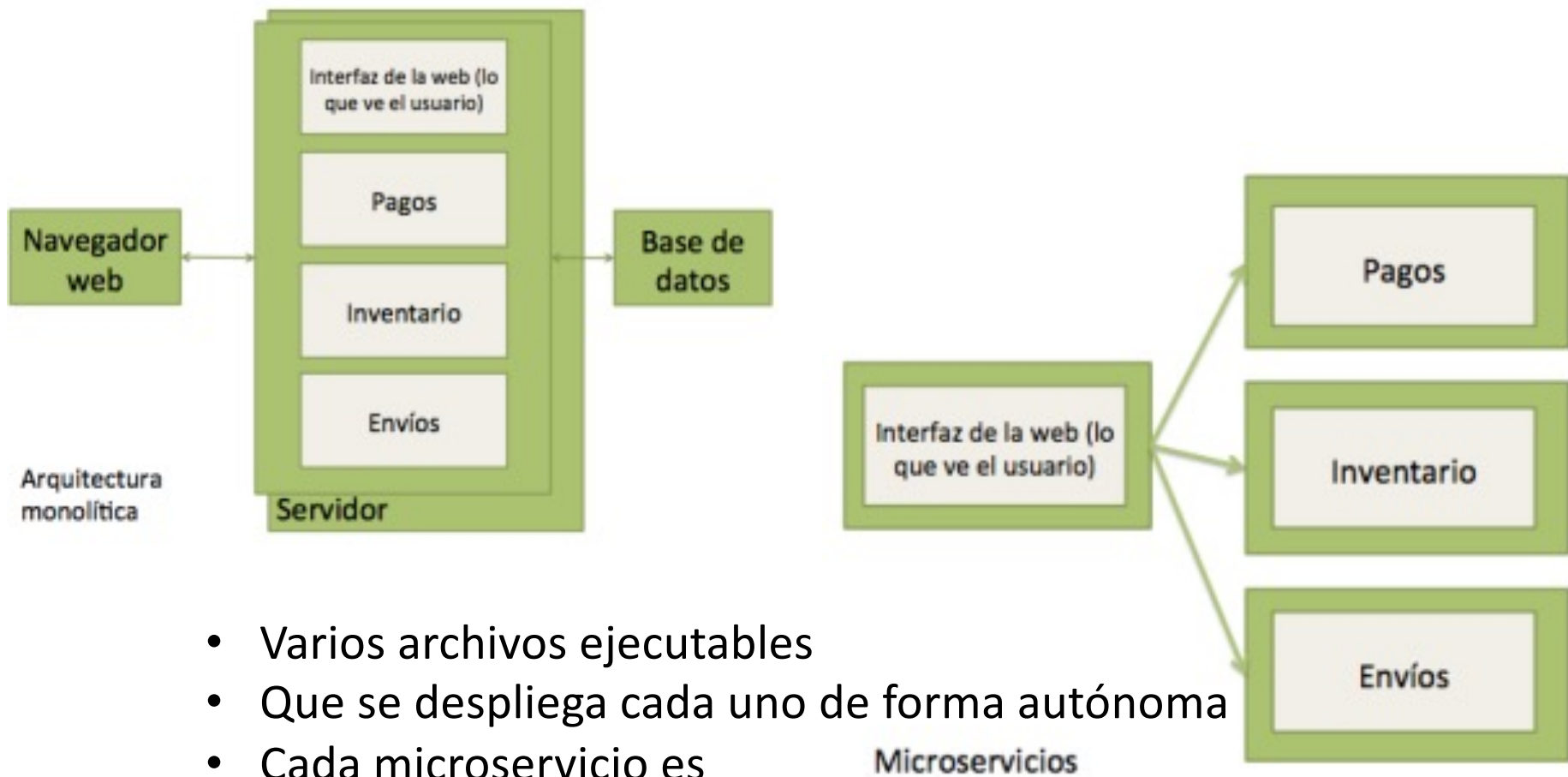


... and scales by distributing these services across servers, replicating as needed.



Del Monolito a Micro-servicios

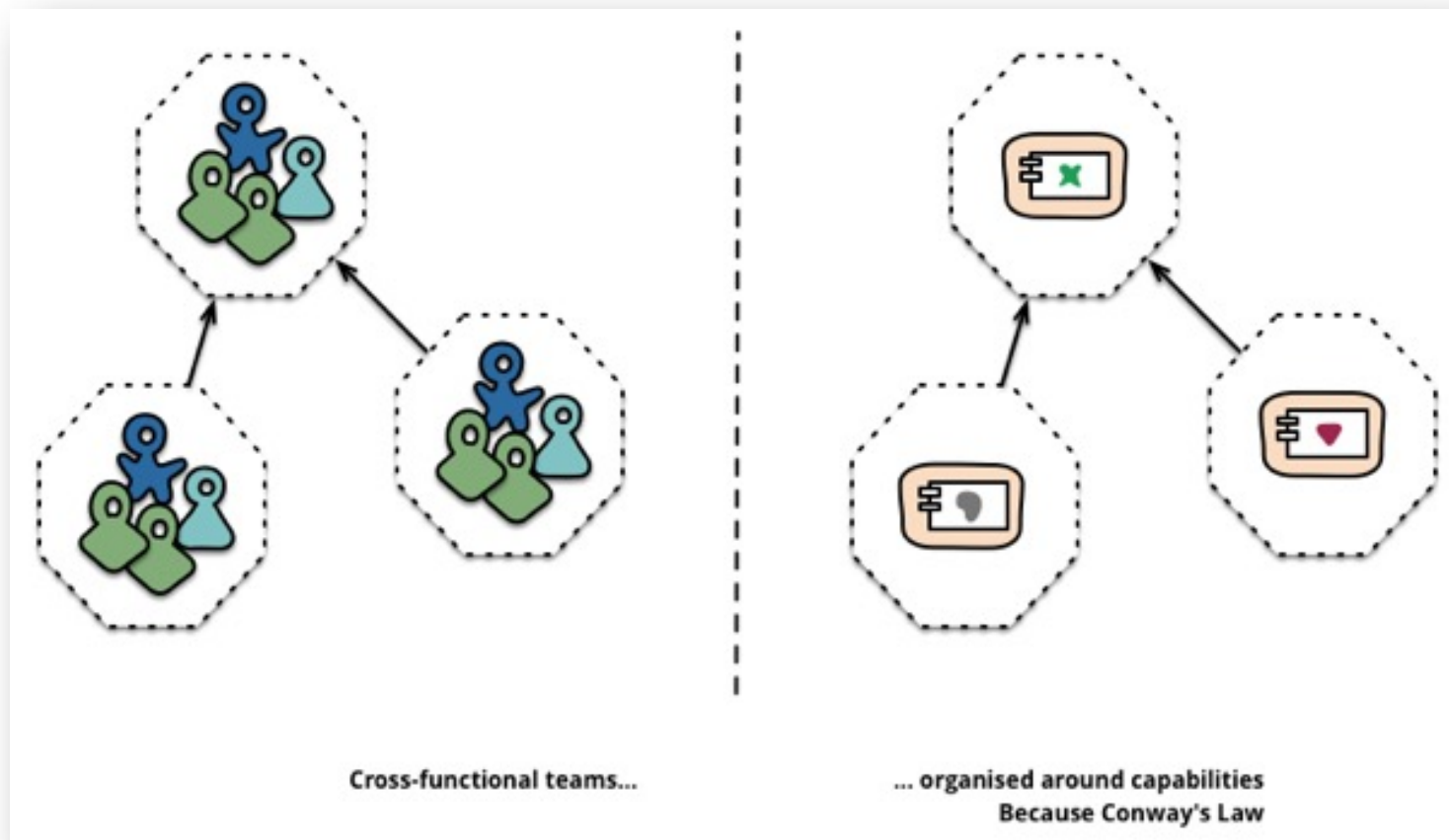
- Un solo archivo ejecutable
- Que se despliega/escala como un todo



- Varios archivos ejecutables
- Que se despliega cada uno de forma autónoma
- Cada microservicio es multi-funcional: *front-end*, *back-end*, *BD*

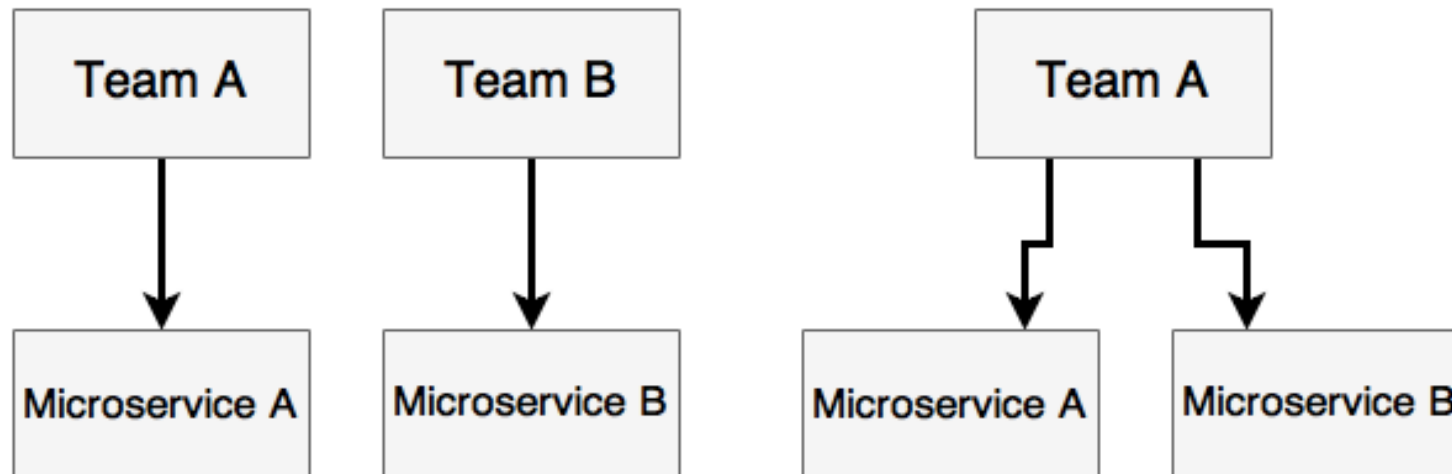
Relación con la Ley de Conway

- Mayor independencia (y responsabilidad) de los equipos respecto a los servicios desarrollados

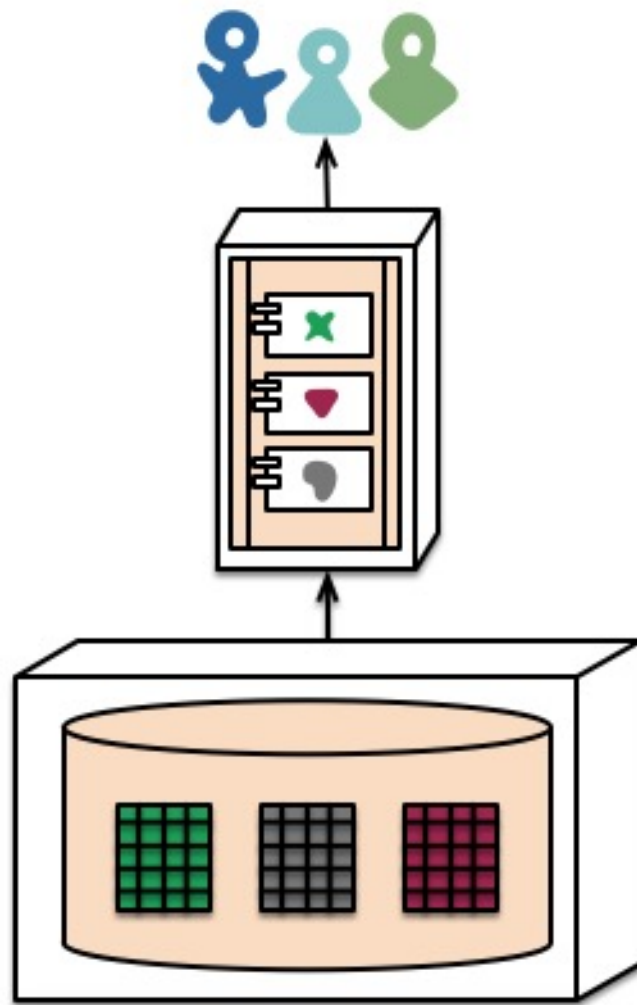


Microservicios y Equipos

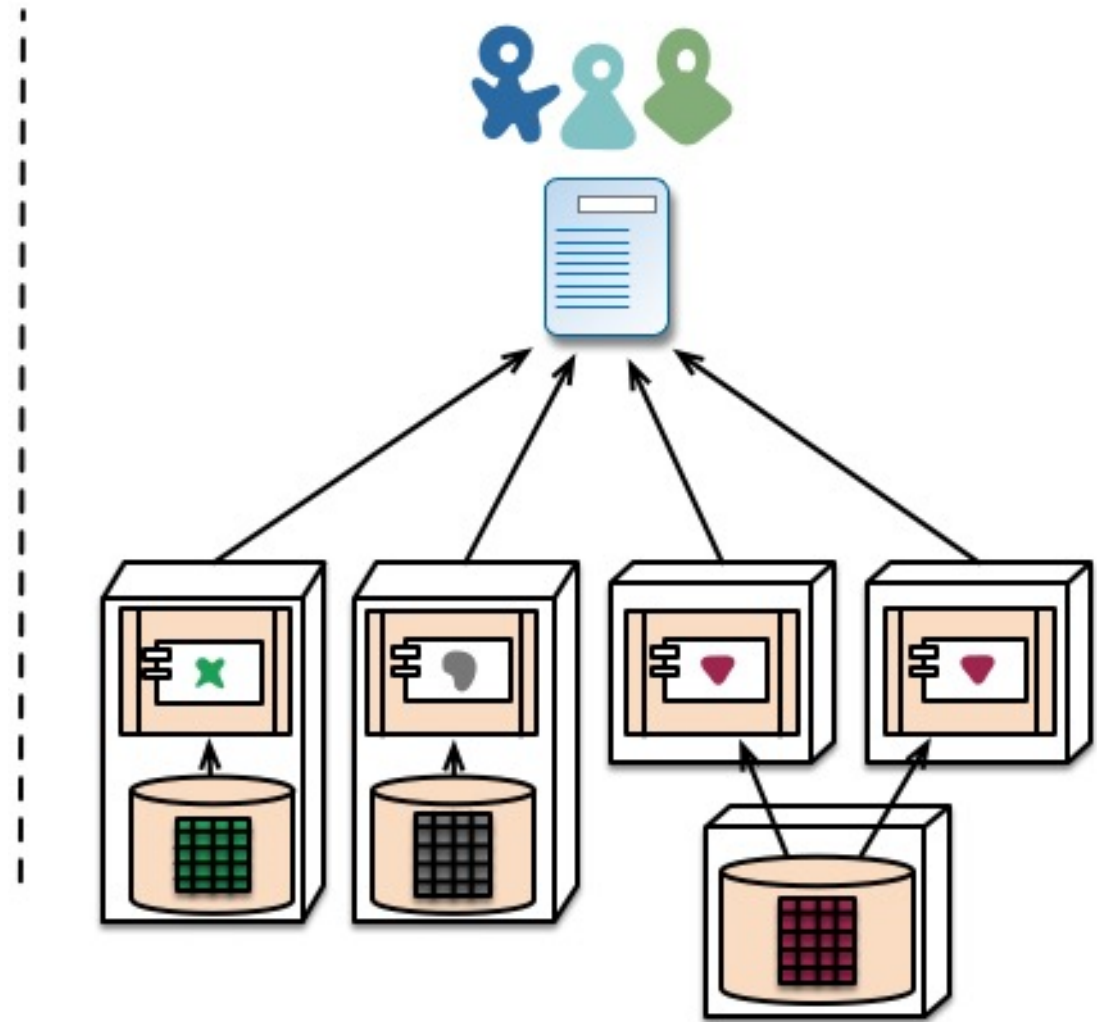
- Cada servicio es propiedad de un único equipo
 - Lo cuál significa que **el equipo es responsable por todo el ciclo de vida del microservicio** → “you build it, you run it”
- La coordinación entre los equipos debiera estar limitada a las responsabilidades implementadas por cada microservicio y a las interfaces del servicio



Gestión de Bases de datos



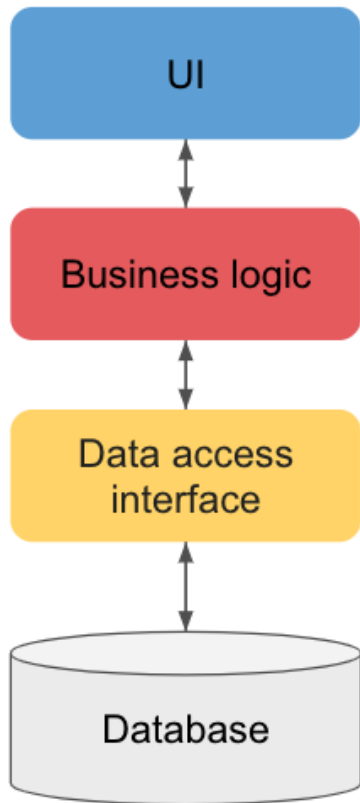
monolith - single database



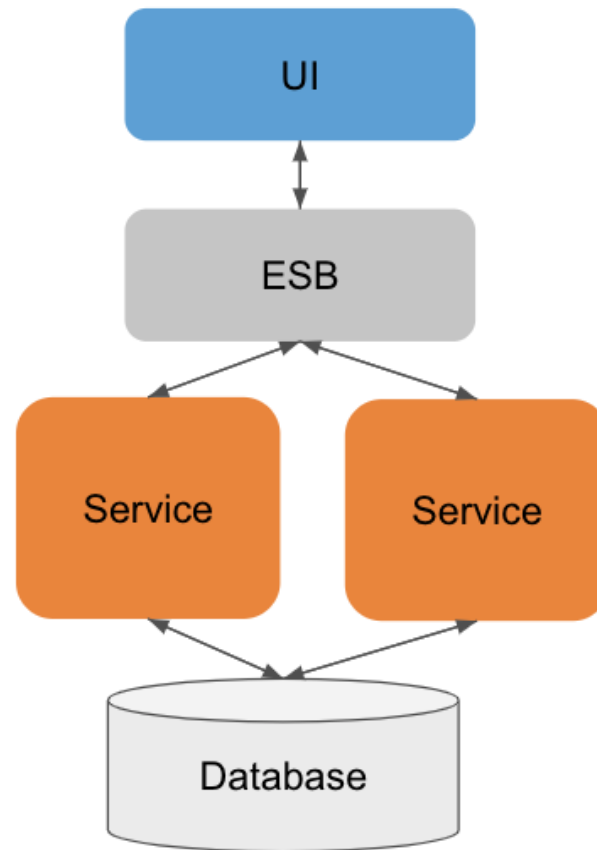
microservices - application databases

Tipos de Arquitecturas

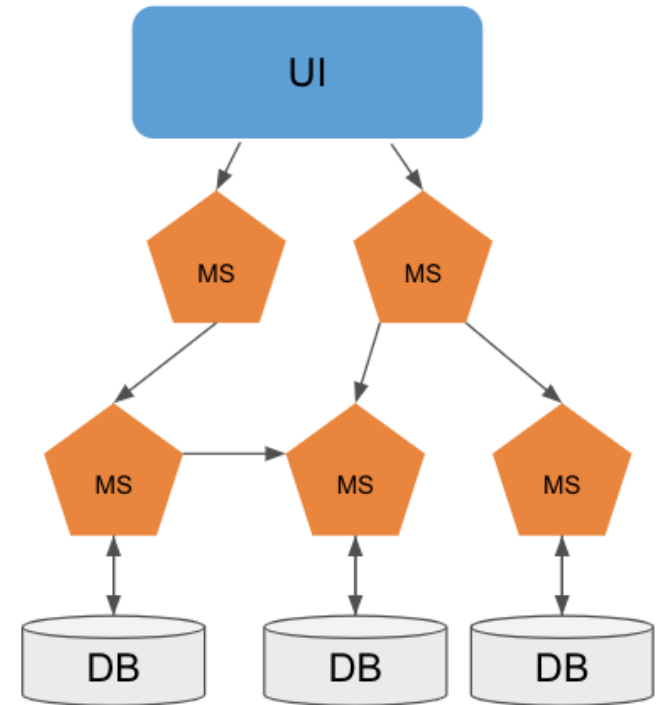
Monolith



SOA



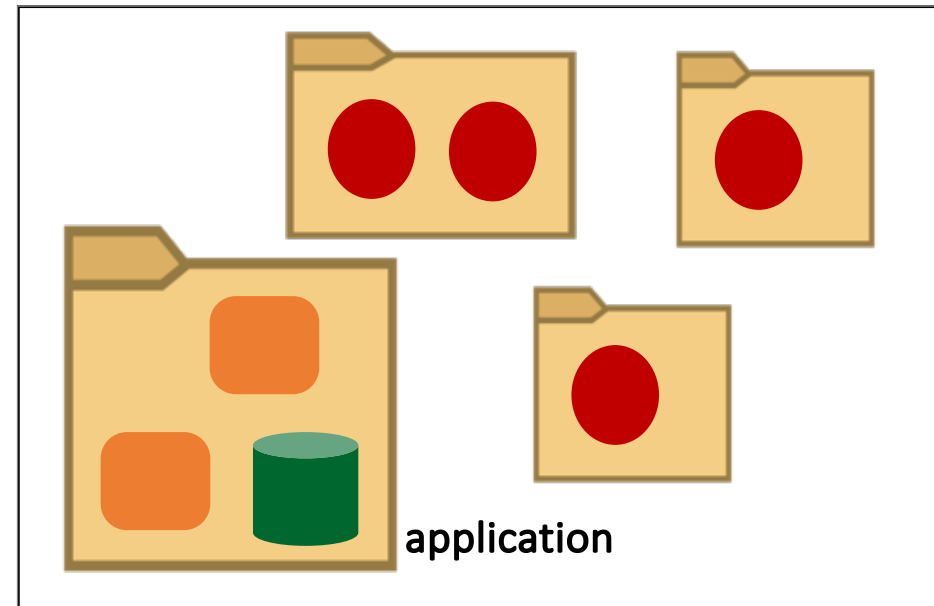
Microservices



Principios de arquitectura

- Una arquitectura de microservicios es **un tipo de arquitectura SOA**
 - Aunque no toda arquitectura SOA tiene que ser de microservicios
- Una arquitectura de microservicios prescribe que **cada unidad de despliegue debe contener solo un servicio o uno pocos servicios cohesivos**
 - La restricción de despliegue es un factor distintivo

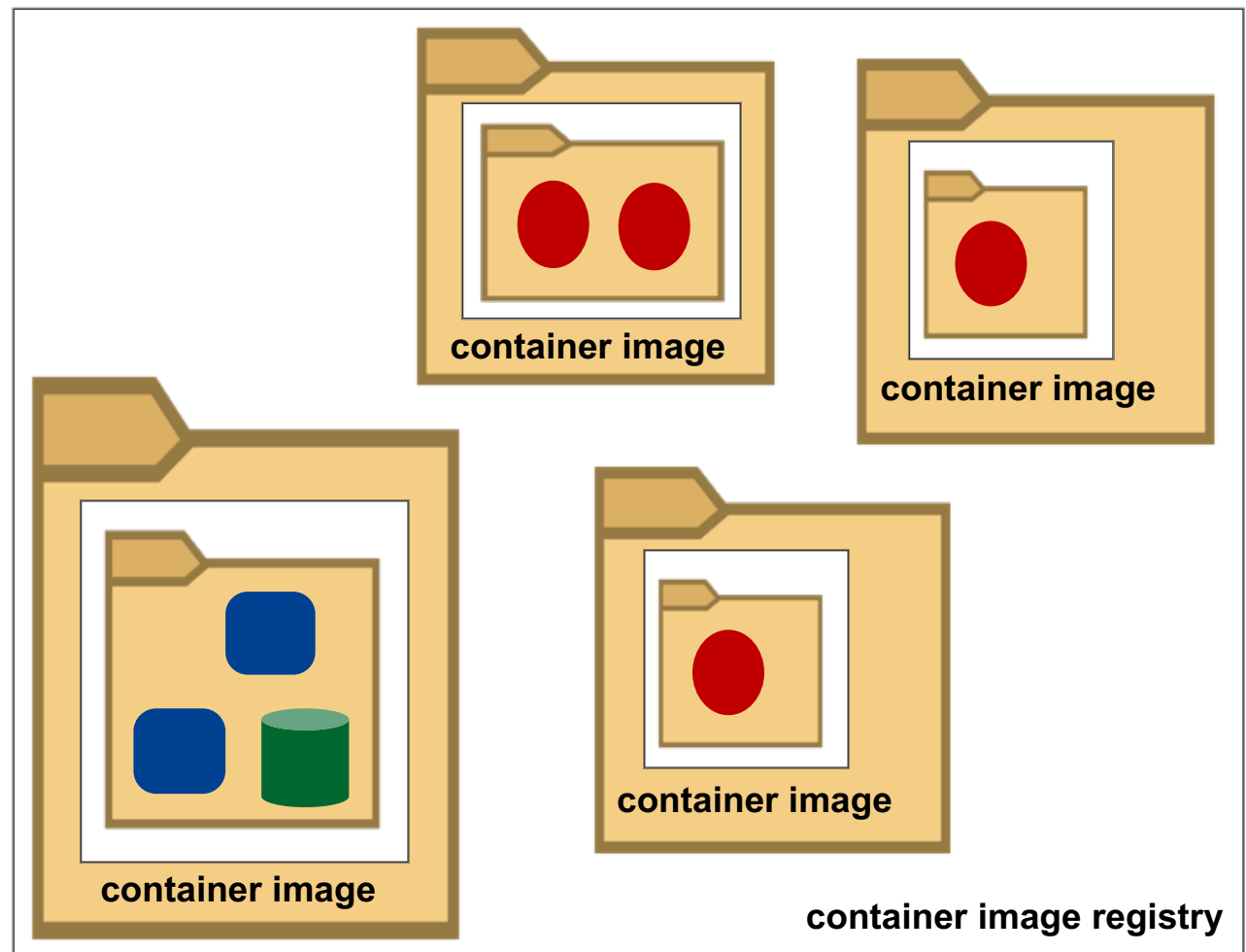
application server



<https://insights.sei.cmu.edu/saturn/2015/11/defining-microservices.html>

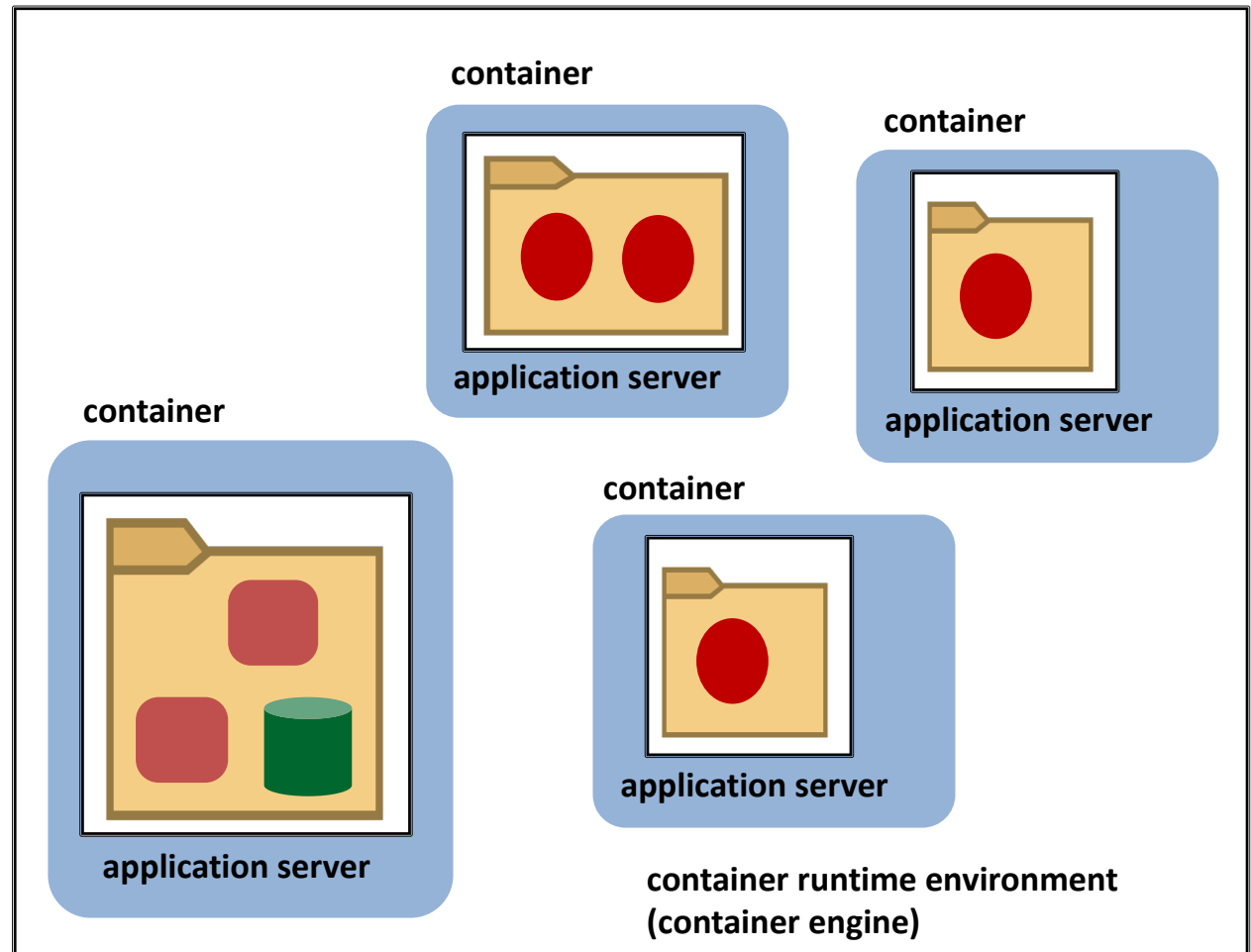
Microservicios y contenedores

- Docker, Kubernetes, Rancher, etc.
- Normalmente se suelen desplegar en ambientes cloud



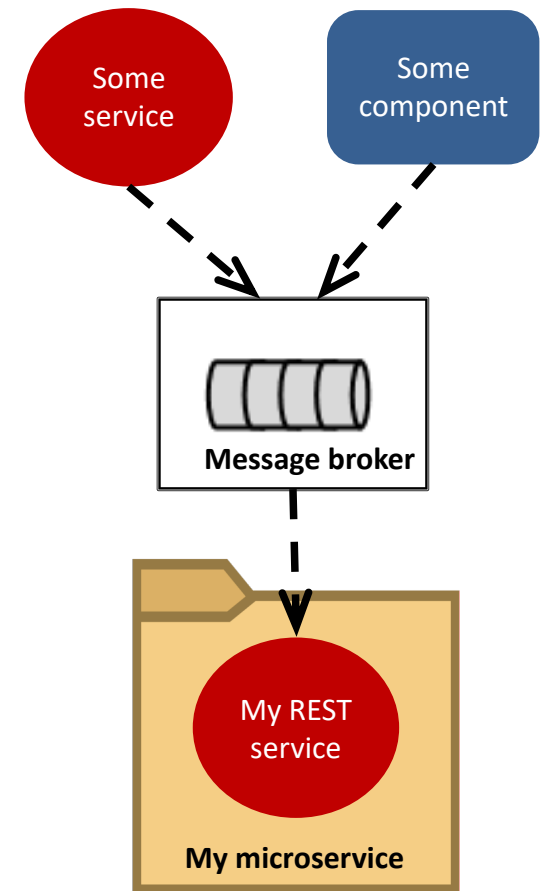
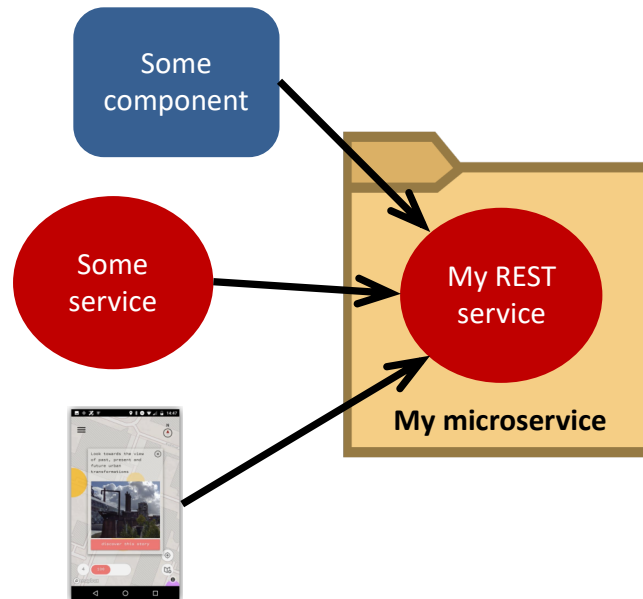
Microservicios y contenedores

- Dockder, Kubernetes, Rancher, etc.
- Normalmente se suelen desplegar en ambientes cloud

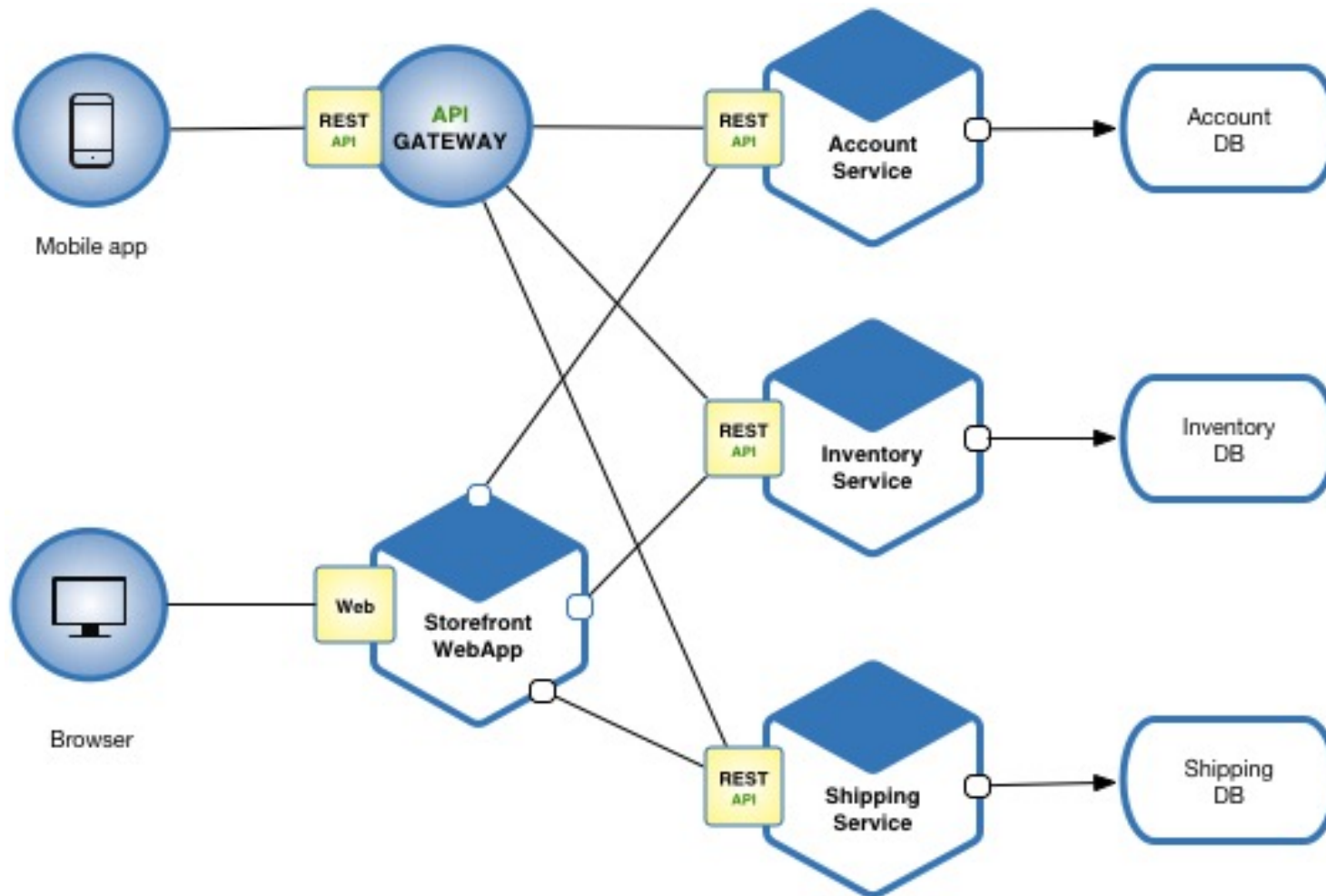


Protocolos de Comunicación

- Los microservicios se comunican sólo mediante mensajes a través de la red
- La comunicación debe ser independiente de la tecnología de implementación de cada microservicio
- Algunos mecanismos comunes son:
 - REST (sobre HTTP)
 - Mensajes/eventos asíncronos

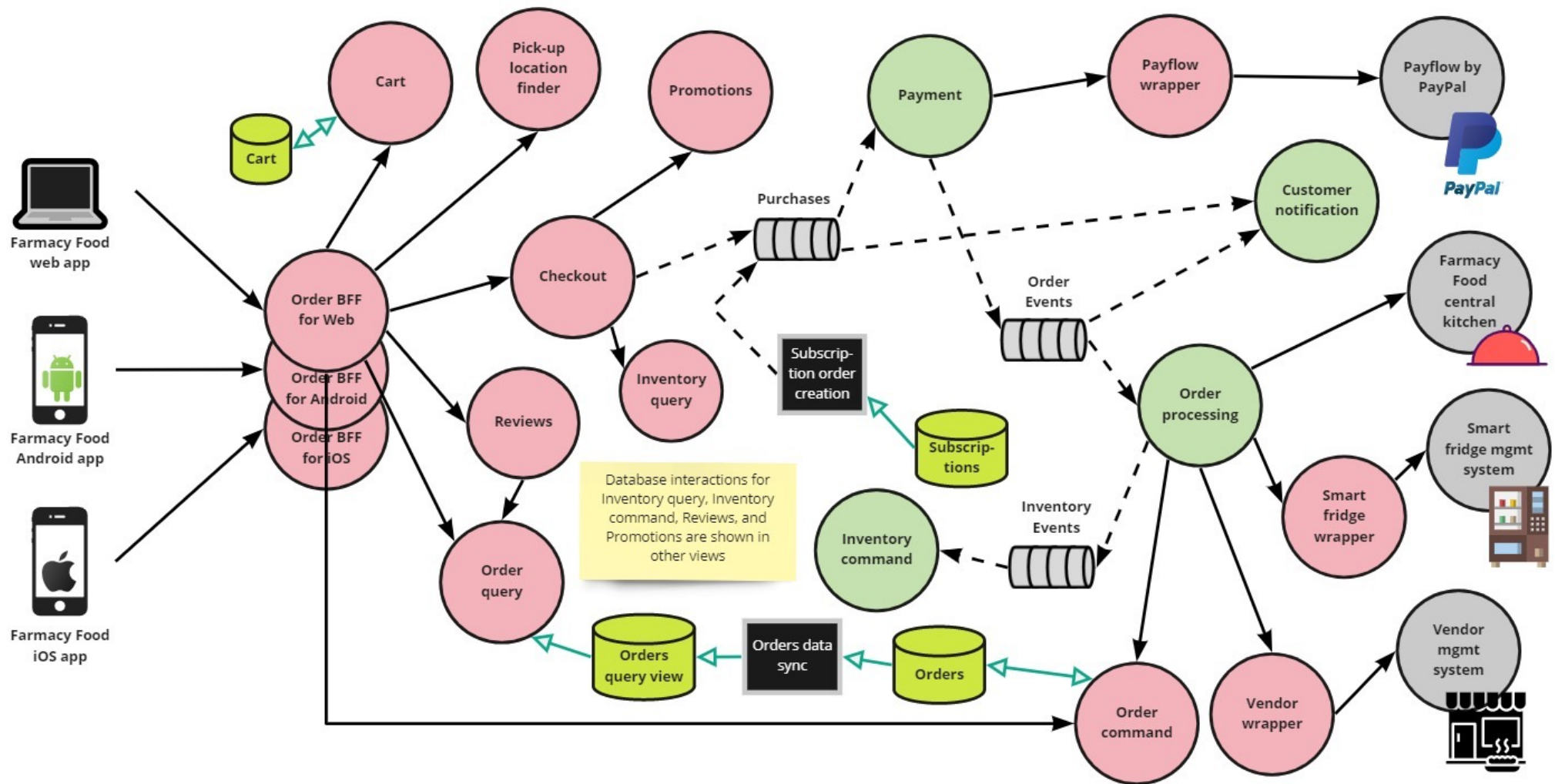


Ejemplo de Aplicación



<https://microservices.io/patterns/microservices.html>

Un Ejemplo más complejo



Microservicios y Atributos de Calidad

- (Como otros estilos arquitectónicos), los microservicios tienden a favorecer ciertos atributos de calidad:
 - Availability and reliability
 - Modifiability
 - Performance
 - Scalability
 - Deployability
 - Service autonomy
 - ...

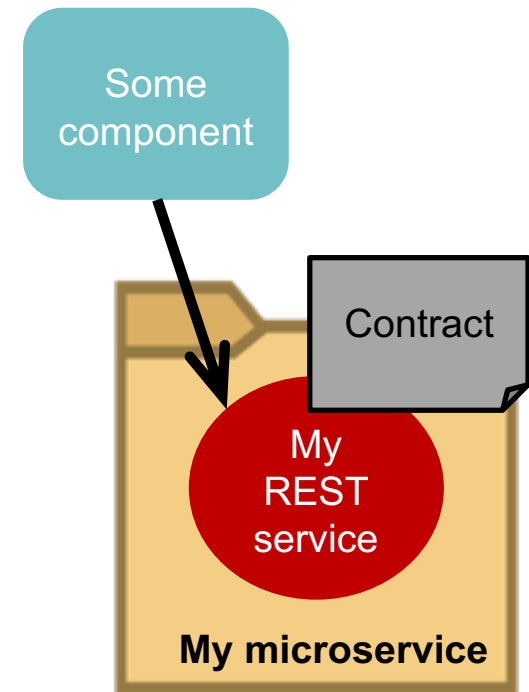


Y estos atributos de calidad pueden ser abordados mediante distintos patrones y tácticas



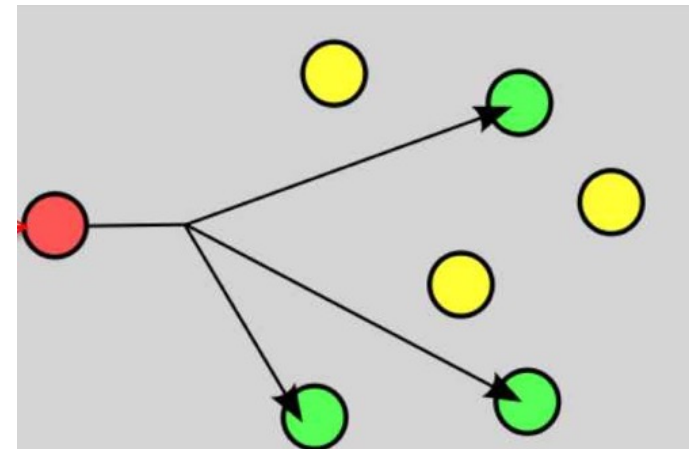
Modificabilidad

- El equipo puede re-diseñar y re-desplegar cada microservicio independientemente
- El equipo tiene libertad para emplear distintos lenguajes, frameworks, bibliotecas y patrones para diseñar e implementar cada microservicio
- Los microservicios son **componentes poco acoplados**, y accesibles solo a través de sus **contratos**
- Sin embargo, el desarrollo de microservicios suele implicar un esfuerzo extra de programación (en comparación a "sin microservicios")



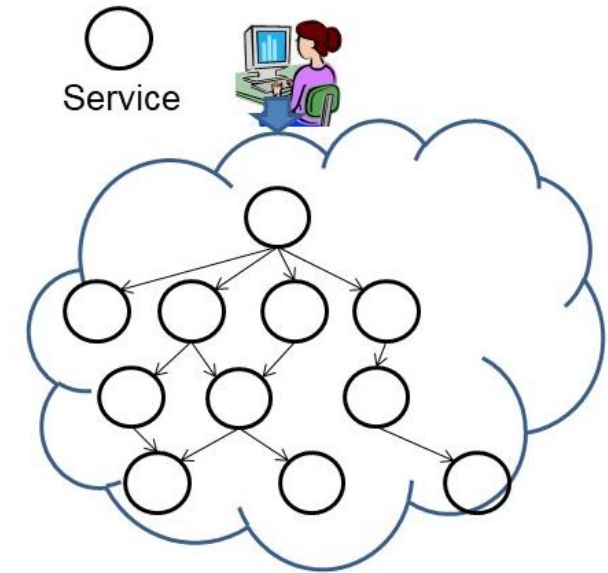
Reusabilidad

- El objetivo es reusar un microservicio dado en otra aplicación
 - Una posible limitación es la dependencia de otros componentes
 - Otro aspecto a considerar es la “cantidad” de funcionalidad asignada a un microservicio (la regla de “las 2 pizzas” es discutible)
- **Fan-out**: el número de servicios “hijos” dependientes de un microservicio que los invoca directamente



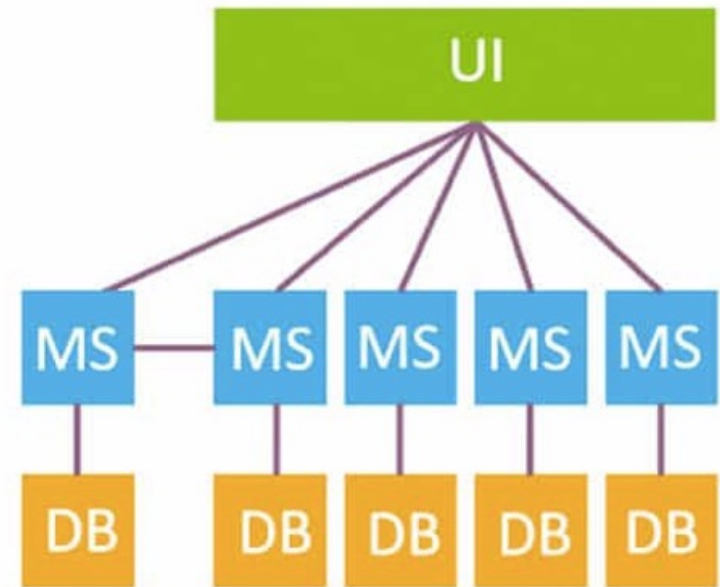
Tipos de Fan-Out

- Fan-out en amplitud (shallow): el microservicio tiene varios hijos (directos) y la cadena de solicitudes (profundidad) es normalmente corta
 - Menos paso de mensajes → menor latencia
- Fan-out en profundidad (deep): el microservicio tiene un número pequeño de hijos (directos), pero la cadena de solicitudes puede ser larga
 - Mayor paso de mensajes → mayor latencia, pero también un mayor potencial de reusabilidad de microservicios

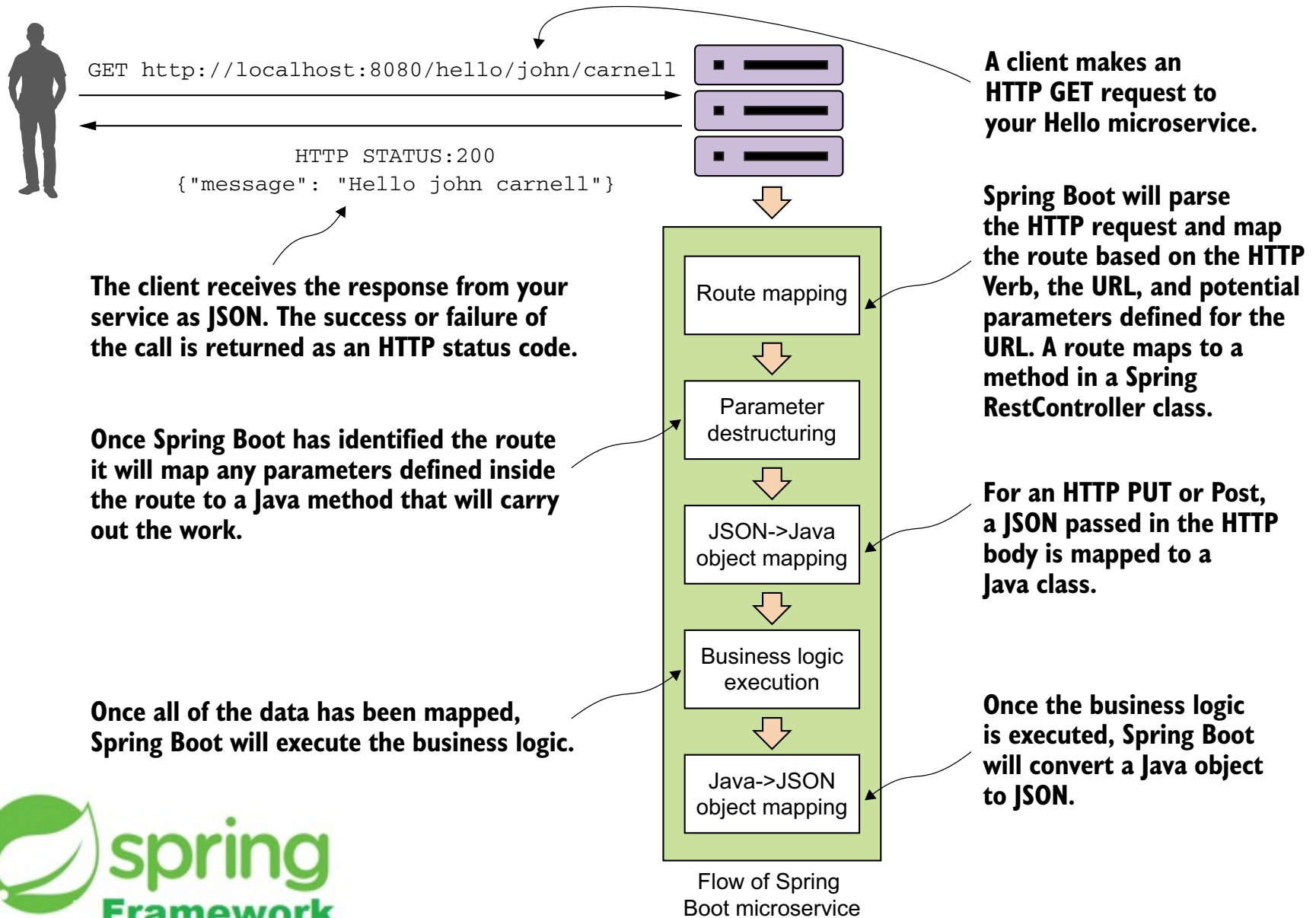


Escalabilidad

- Cada microservicio puede ser escalado independientemente, utilizando pools, clusters y grids (entre otros mecanismos)
 - **Escalado horizontal**, que generalmente involucra contenedores o VMs
- Las facilidades de despliegue de los microservicios hacen que tengan una buena alineación con la **elasticidad** de los ambientes de **cloud computing**

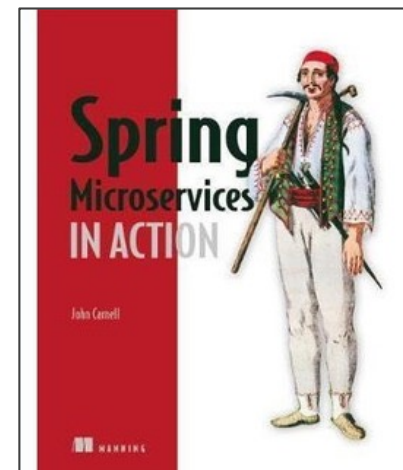


Relación con Spring

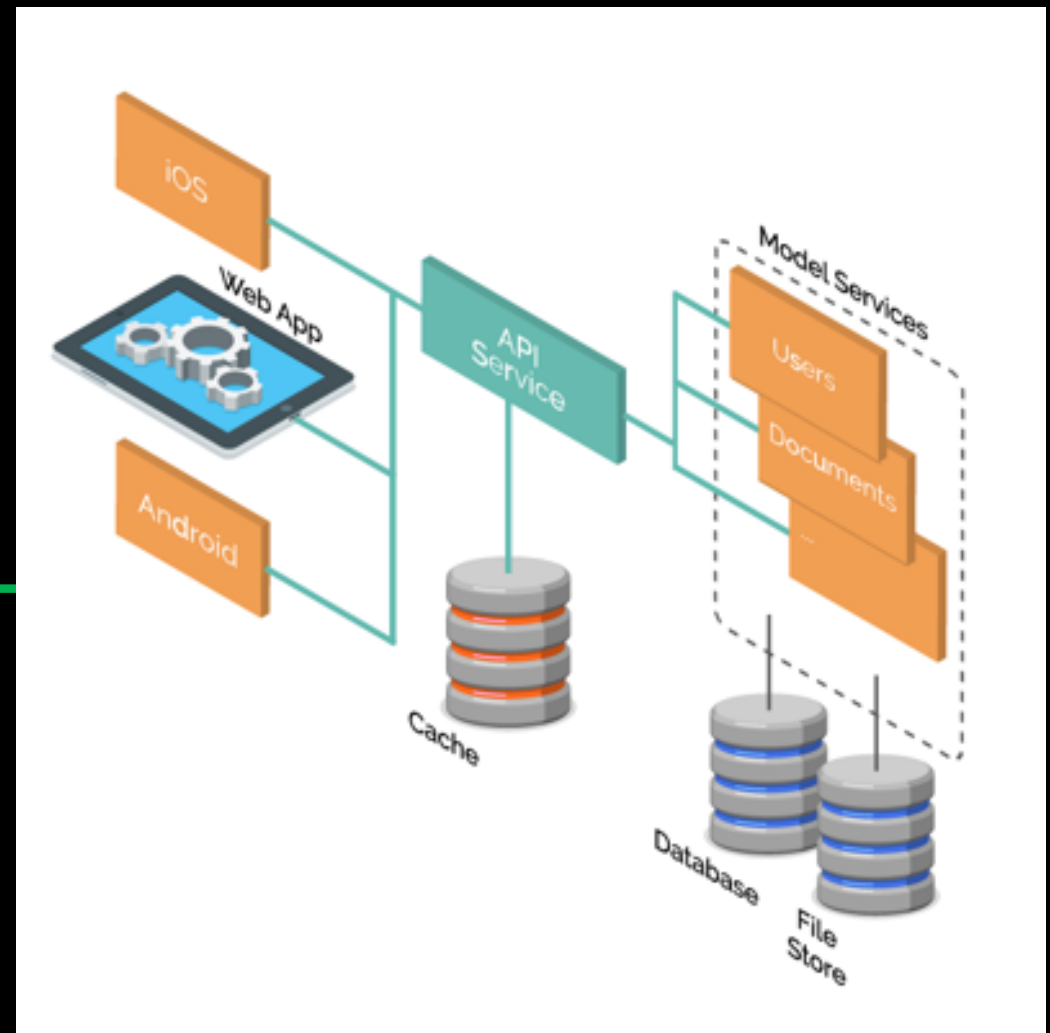


Desarrollo de Microservicios

- Descomposición del problema y **modelamiento/diseño** en términos de entidades y servicios
- Granularidad de cada uno de los servicios
- Protocolos de comunicación
- Diseño de las interfaces
 - Endpoints a exponer
- Gestión de configuración y despliegue
- Procesamiento de eventos
 - Por ej., para comunicar cambios de estado o datos
- Descubrimiento y ruteo de servicios
- Balanceo de servicios y resiliencia
- Seguridad
- Logging y trazabilidad
- ...

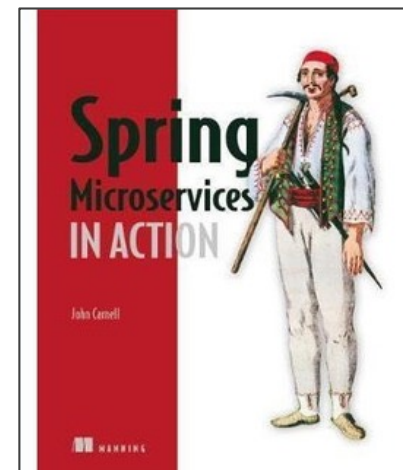


Algunos patrones

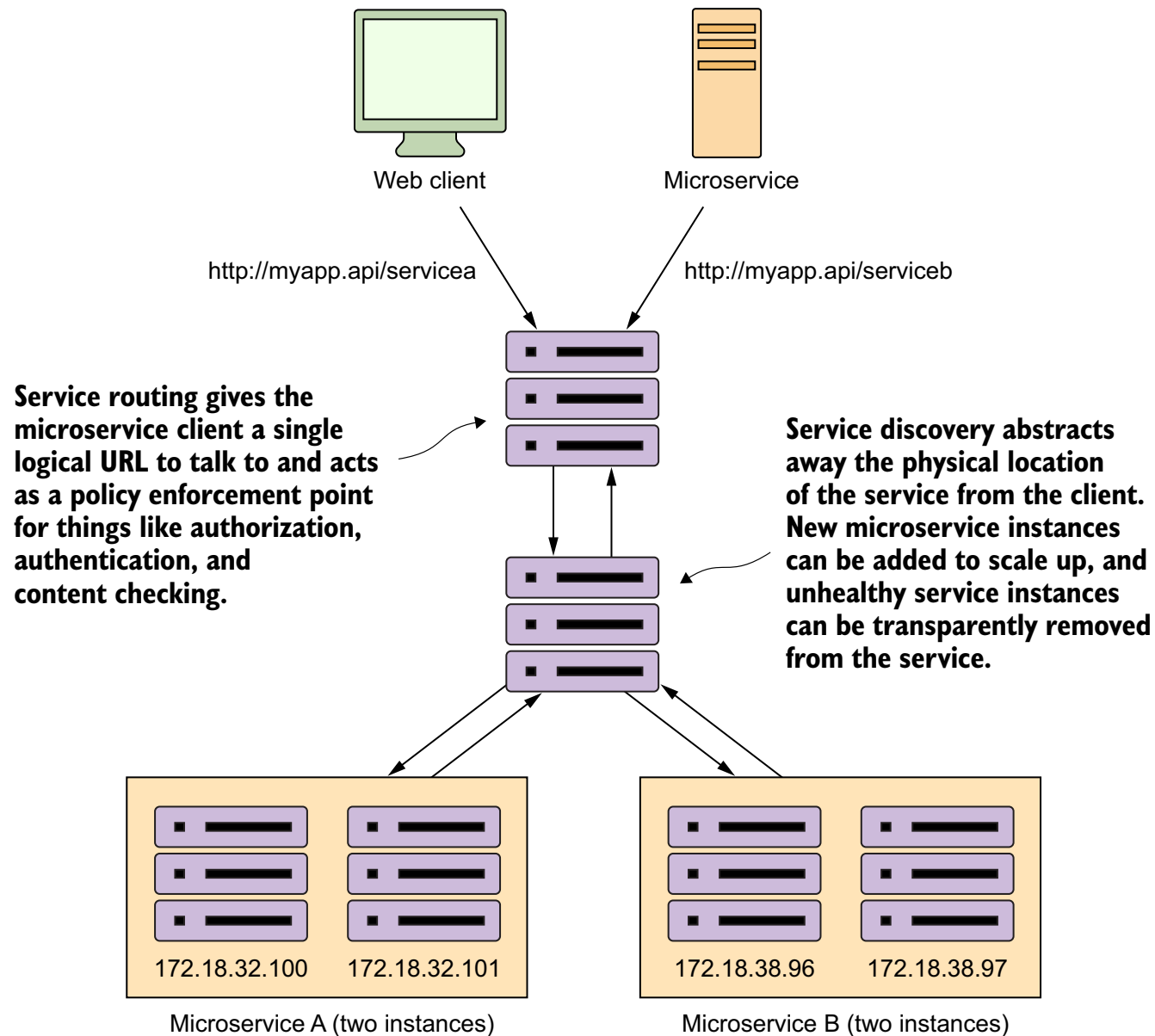


Desarrollo de Microservicios

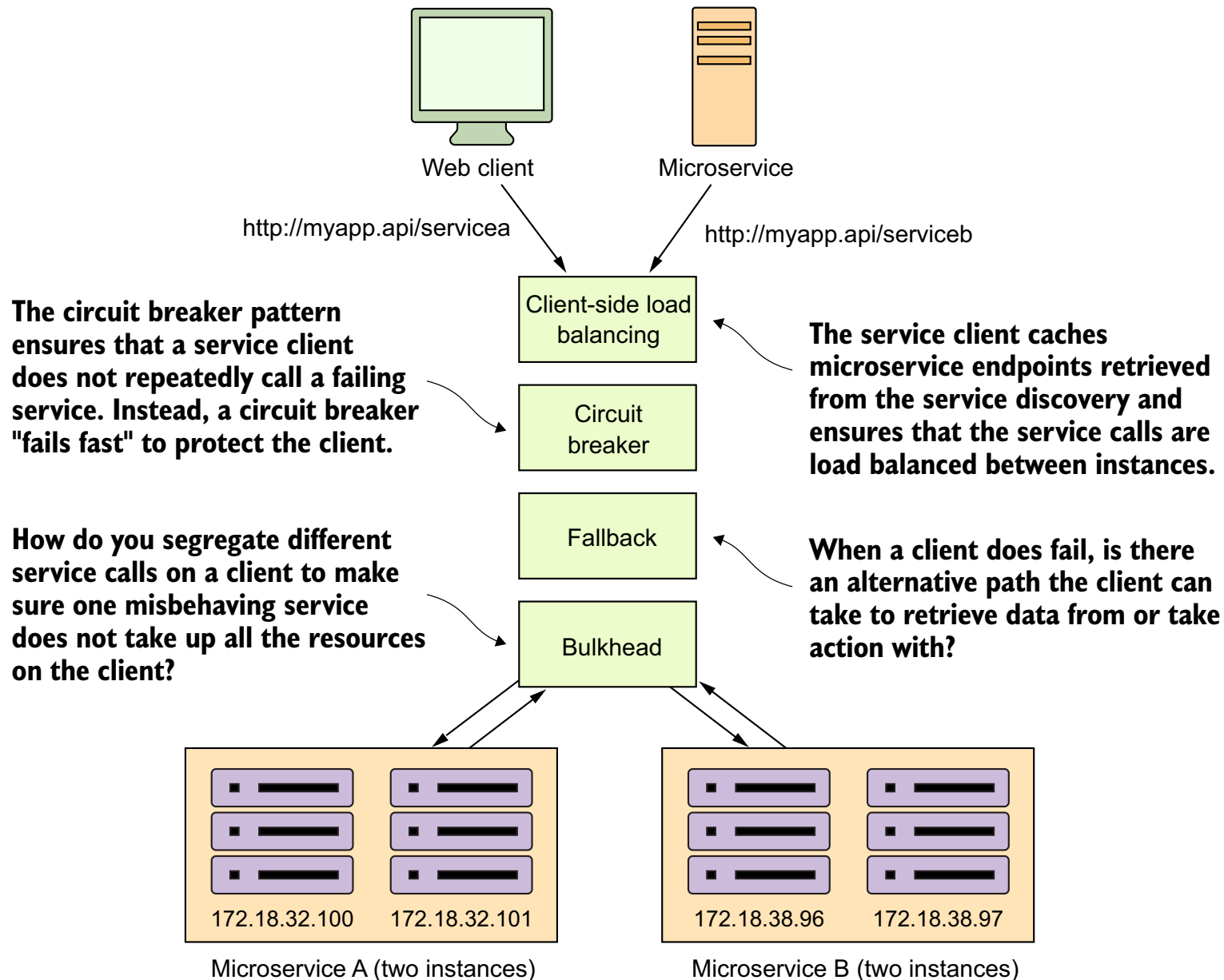
- Descomposición del problema y **modelamiento/diseño** en términos de entidades y servicios
- Granularidad de cada uno de los servicios
- Protocolos de comunicación
- Diseño de las interfaces
 - Endpoints a exponer
- Gestión de configuración y despliegue
- Procesamiento de eventos
 - Por ej., para comunicar cambios de estado o datos
- Descubrimiento y ruteo de servicios
- Balanceo de servicios y resiliencia
- Seguridad
- Logging y trazabilidad
- ...



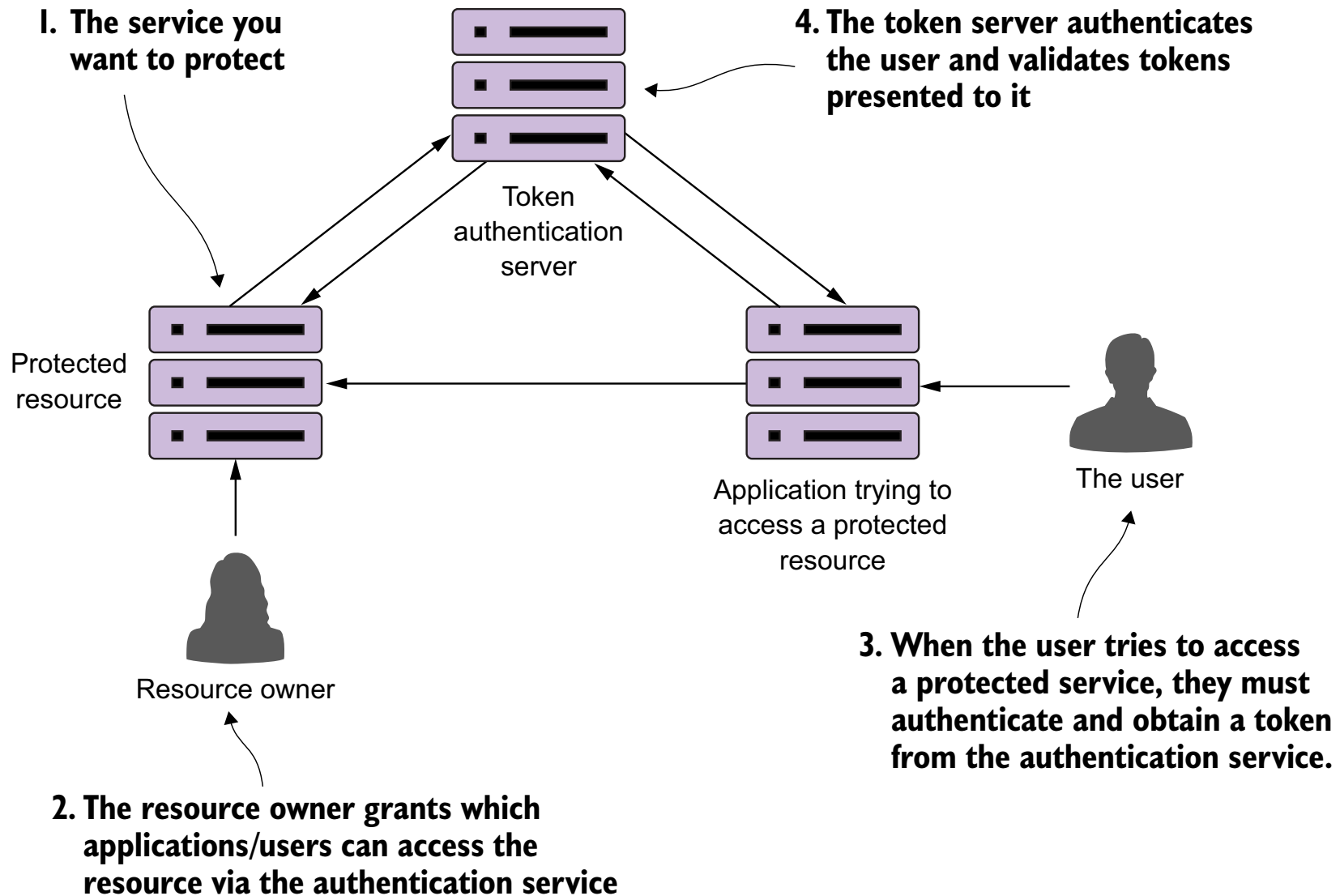
Ruteo y Descubrimiento



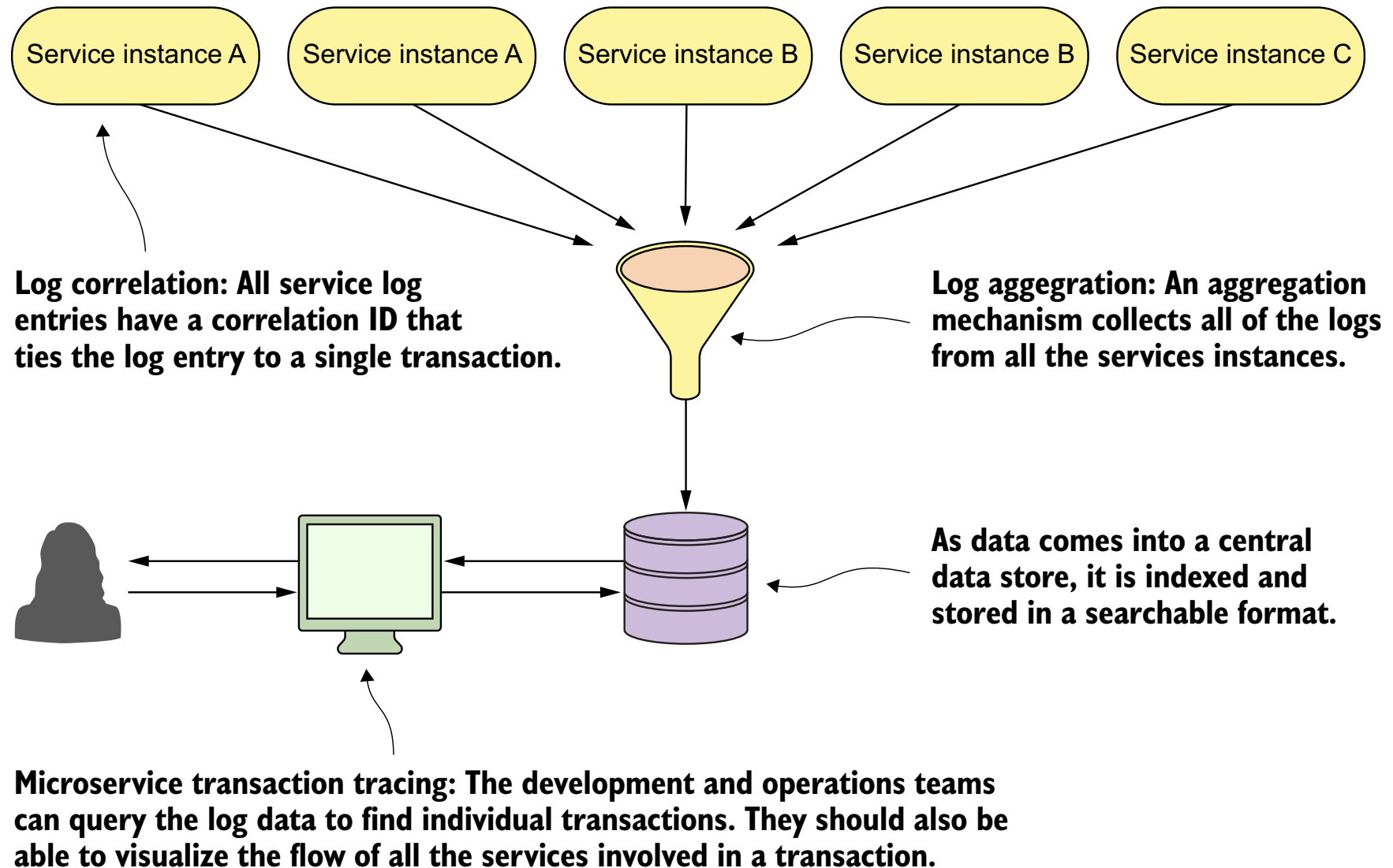
Resiliencia



Seguridad



Logging y Trazabilidad



Estandarización de Servicios REST

- Uso de URIs descriptivas. Ejemplo:
 - www.myweather.com/avg/{city}/{month} [Not OK]
 - www.myweather.com/averages/city/{city}/month/{month} [OK]
- Uso de parámetros en queries para:
 - Paginación. Ejemplo: [/forecast/monthly/{city}?page=1](#)
 - Filtrado de atributos. Ejemplo: [/forecast/monthly/{city}?attributes=max,min](#)
 - Criterios de búsqueda. Ejemplo: [/severeweatheralerts?year=2015&month=9](#)
- Elección de use-of-hyphen por sobre use_of_underscore
 - Alternativas: useOfCamelCase, lowercaseonly
- Usar ya sea singular o plural para los sustantivos, de forma consistente



SWAGGER



Estandarización de Servicios REST

- Usar métodos HTTP para indicar la acción a realizar. Ejemplo:
 - <http://www.myweather.com/getCurrentTemp/zip/{zipCode}> [Not OK]
 - GET <http://www.myweather.com/currentTemp/zip/{zipCode}> [OK]
- Elegir qué método HTTP utilizar en cada situación. Ejemplos:

HTTP method	When to use it
get	Queries and read-only operations (no data should be modified)
put	Updates on resources in an idempotent fashion
delete	Removal of resources in an idempotent fashion
post	Resource creation; or non-idempotent updates or other operations on resources
patch	Partial updates (i.e., only part of the resource is updated)



Estandarización de Servicios REST

HTTP status codes para respuestas:

- *200: success*
- *201: successful resource creation*
- *202: successful start of a request that will be handled asynchronously*
- *400: data validation error or request processing error*
- *401: invalid user credentials provided for authentication*
- *403: access denied (authorization failure)*
- *404: the requested resource was not found*
- *409: the request creates a conflict (e.g., duplicate key on resource creation)*
- *415: the request content-type header is an unsupported media type*
- *500: unexpected error processing the request*



Recapitulación

- Un **(micro)servicio** debe ser lo suficientemente “pequeño” como para ser desarrollado por un único equipo y ser fácilmente testeable
 - **Principio de Responsabilidad Unica (POO)**, en el sentido que una clase o conjunto de clases deben tener una funcionalidad cohesiva y bajo acoplamiento con otras clases (que implementan otras responsabilidades)
 - El servicio debe “encapsular” cambios relacionados a su responsabilidad
- Un microservicio no solo involucra su código fuente (por ej., SpringBoot) sino la **capacidad de ser desplegado independientemente** de otros microservicios → runtime



Catálogo *microservices.io*

Microservice Architecture

Supported by Kong

About Microservices.io



Microservices.io is brought to you by Chris Richardson. Experienced software architect, author of POJOs in Action and the creator of the original [CloudFoundry.com](#). His latest startup is [eventuate.io](#), a microservices application platform.

Public training in Minneapolis

I'll be teaching a 2 day microservices workshop in Minneapolis in January and then keynoting at the CodeFreeze conference.

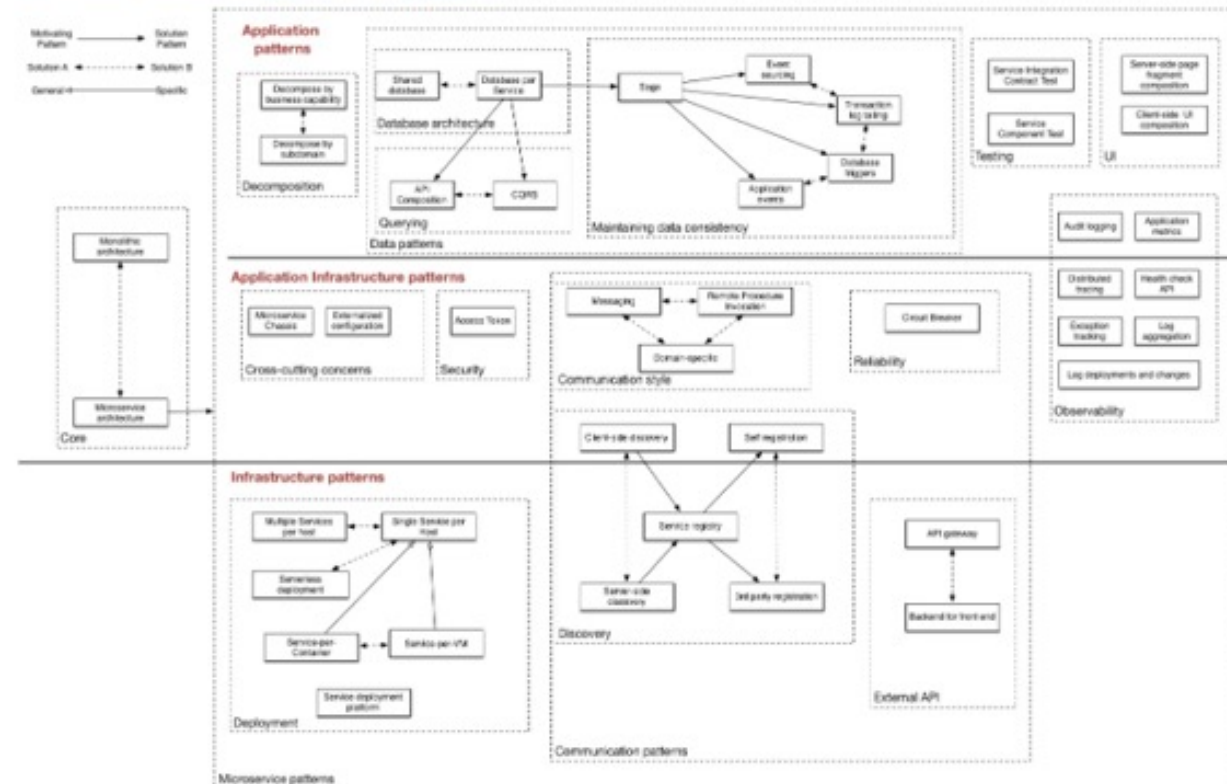
[Patterns](#)[Articles](#)[Presentations](#)[Resources](#)[Assessment Platform](#) **new**[Other Languages](#)[About](#)

A pattern language for microservices

The beginnings of a [pattern language](#) for microservice architectures.

[点击这里，访问本系列文章的中文翻译](#)

[Click here for Chinese translation of the patterns](#)



Core patterns

The patterns

How to apply the patterns

Core patterns

- [Monolithic architecture](#)
- [Microservice architecture](#)

Decomposition

- [Decompose by business capability](#)
- [Decompose by subdomain](#)

Deployment patterns

- [Multiple service instances per host](#)
- [Service instance per host](#)
- [Service instance per VM](#)
- [Service instance per Container](#)
- [Serverless deployment](#)
- [Service deployment platform](#)

Cross cutting concerns

- [Microservice chassis](#)
- [Externalized configuration](#)

Communication style

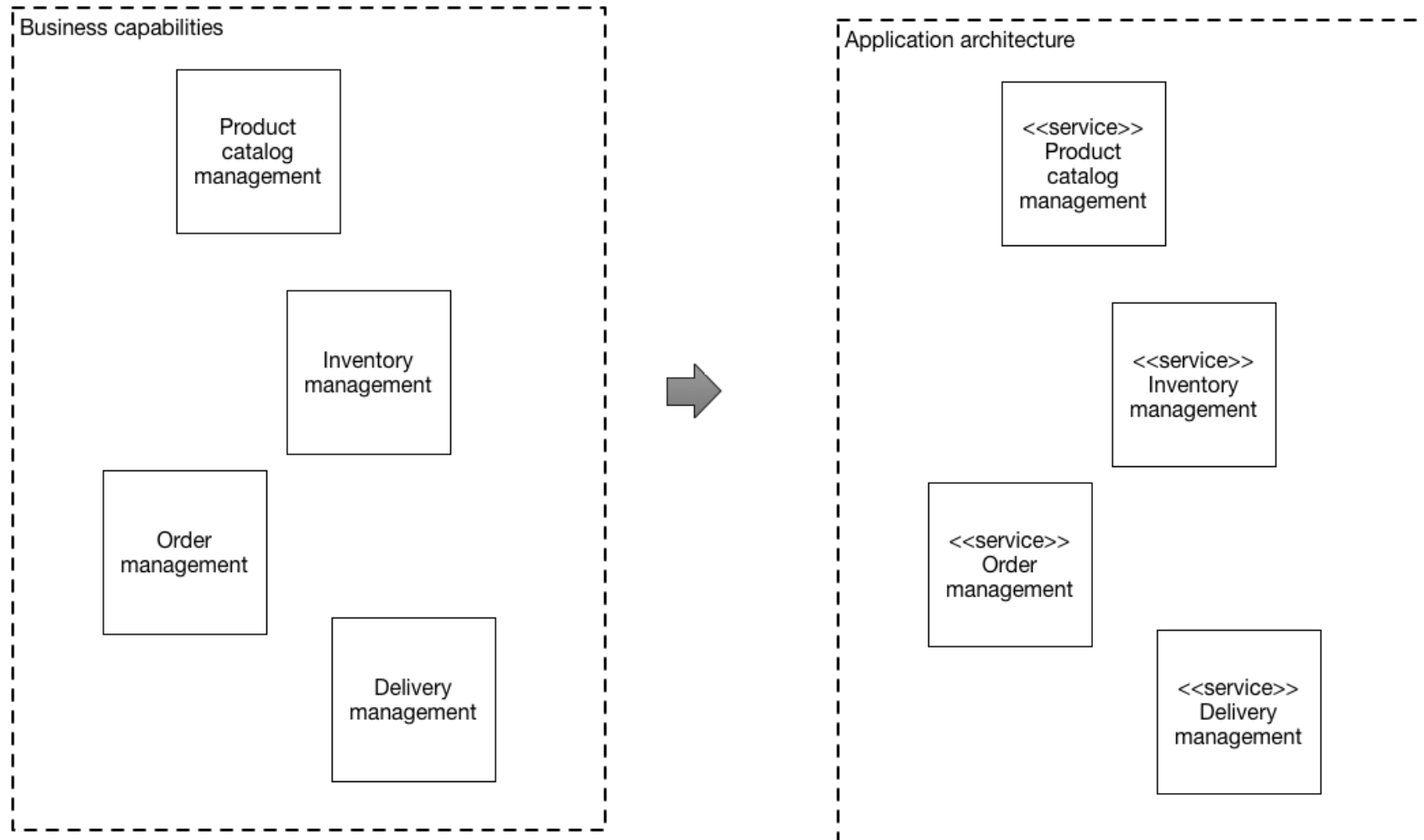
- [Remote Procedure](#)

Modelado de Microservicios

- Dos estrategias
 1. Descomponer el problema en base a **capacidades del negocio**
 2. Descomponer el problema en base a **sub-dominios**
- Mas generalmente, se aplica un enfoque denominado **DDD (Domain-Driven Design)**

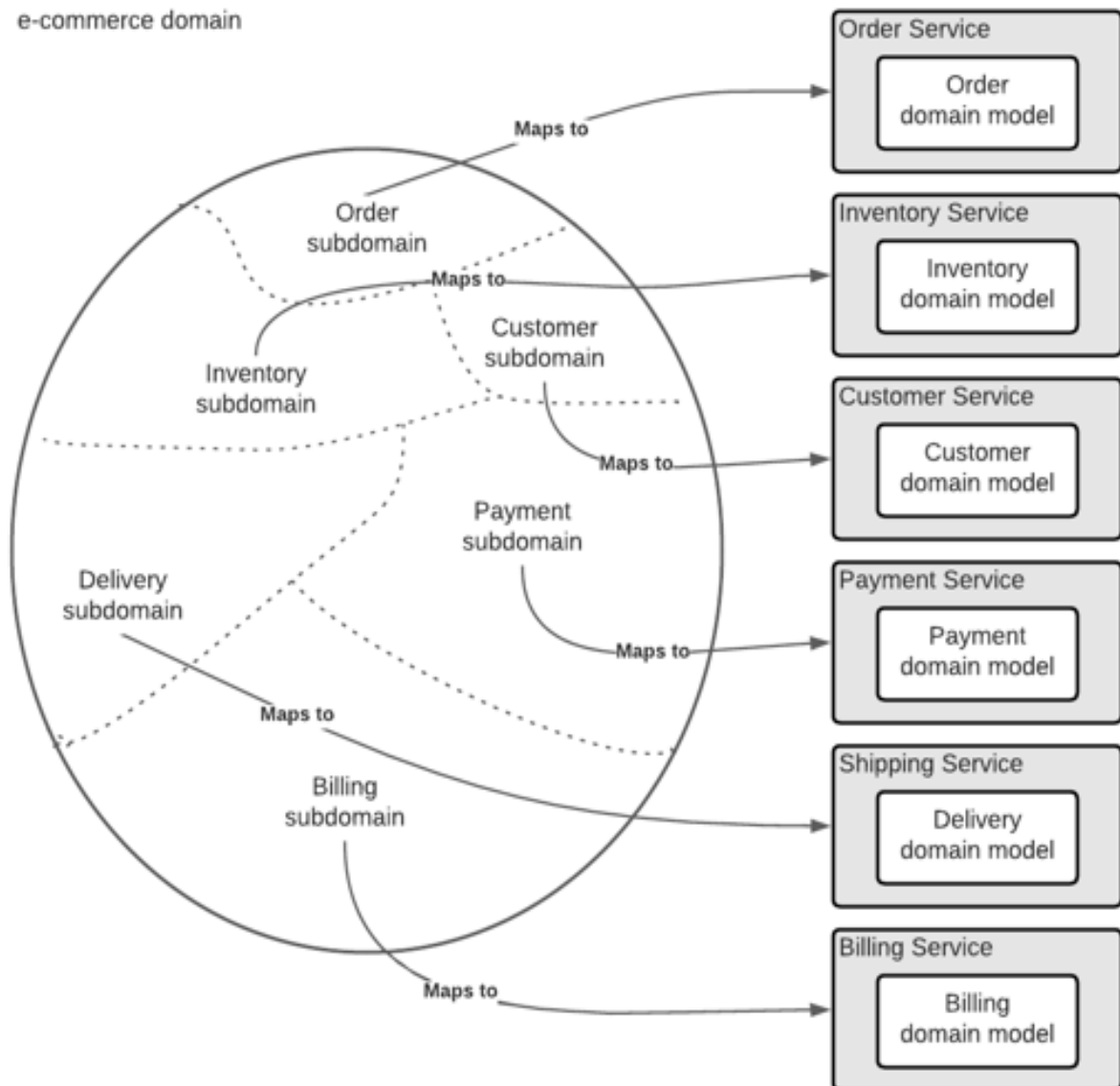


Descomposición por Capacidades de Negocio

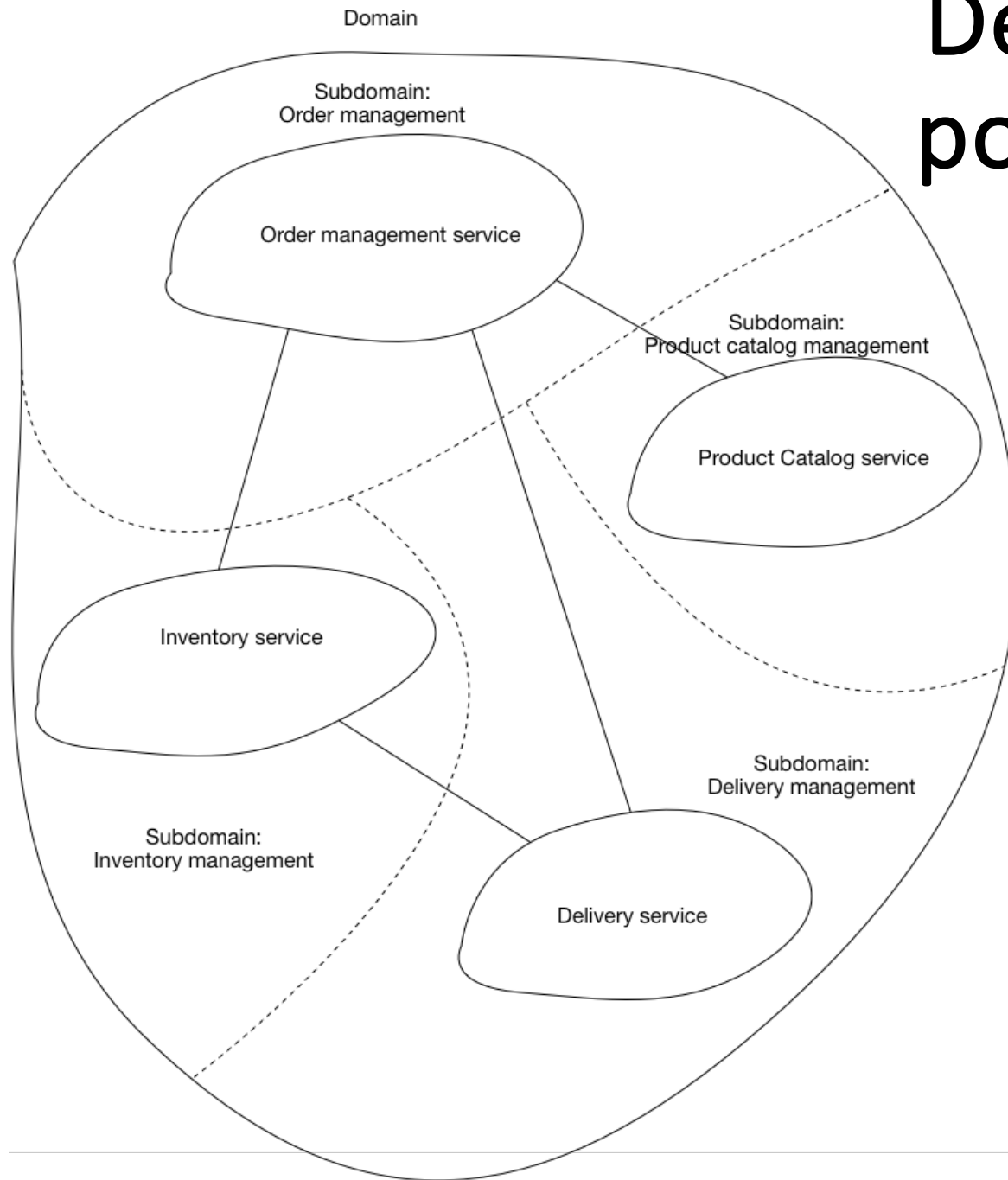


Domain-driven Design (DDD)

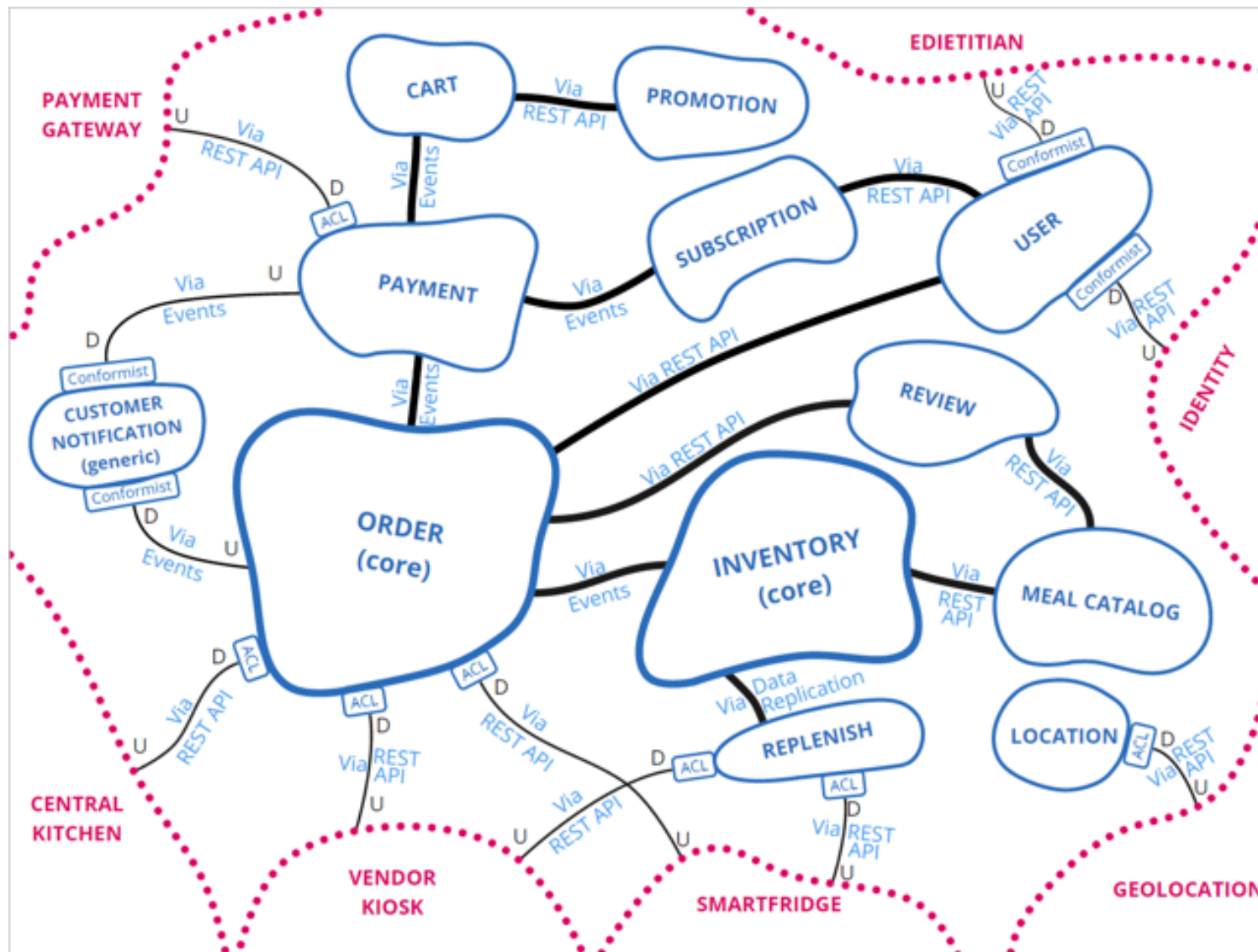
- Es una técnica de modelamiento que captura “dominios” y servicios asociados, que luego progresivamente sirven para derivar los microservicios del sistema



Descomposición por Sub-dominio



Descomposición por Sub-dominio



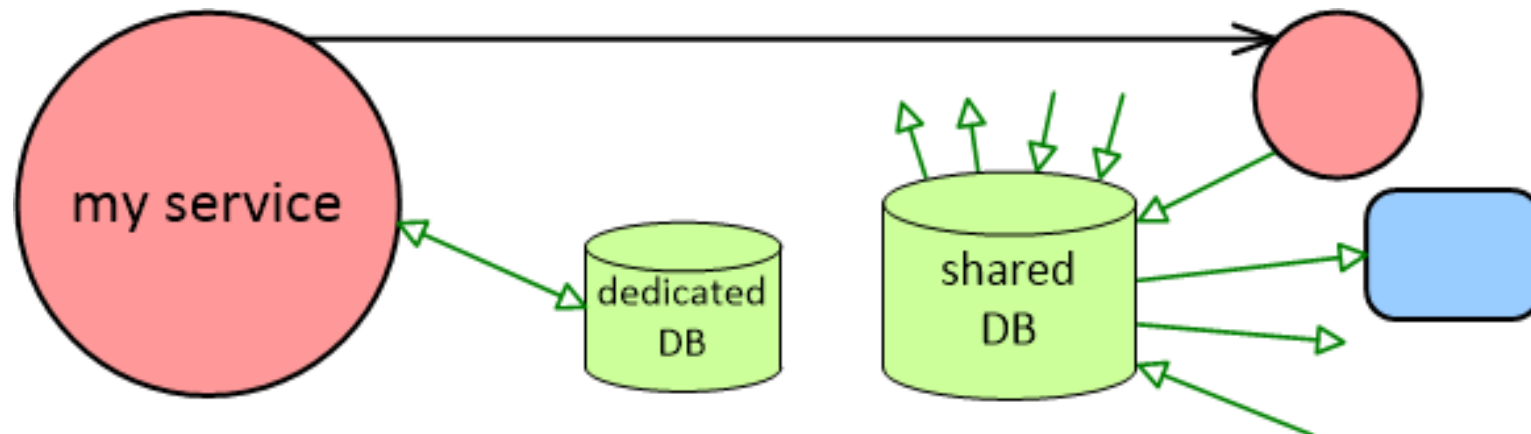
Patrones más técnicos

- Database per microservice
- Shared database
- Wrapper service
- Service data replication
- Gateway
- EDA (Event-driven Architecture)
 - Orquestación
 - Coreografía
- Circuit-breaker

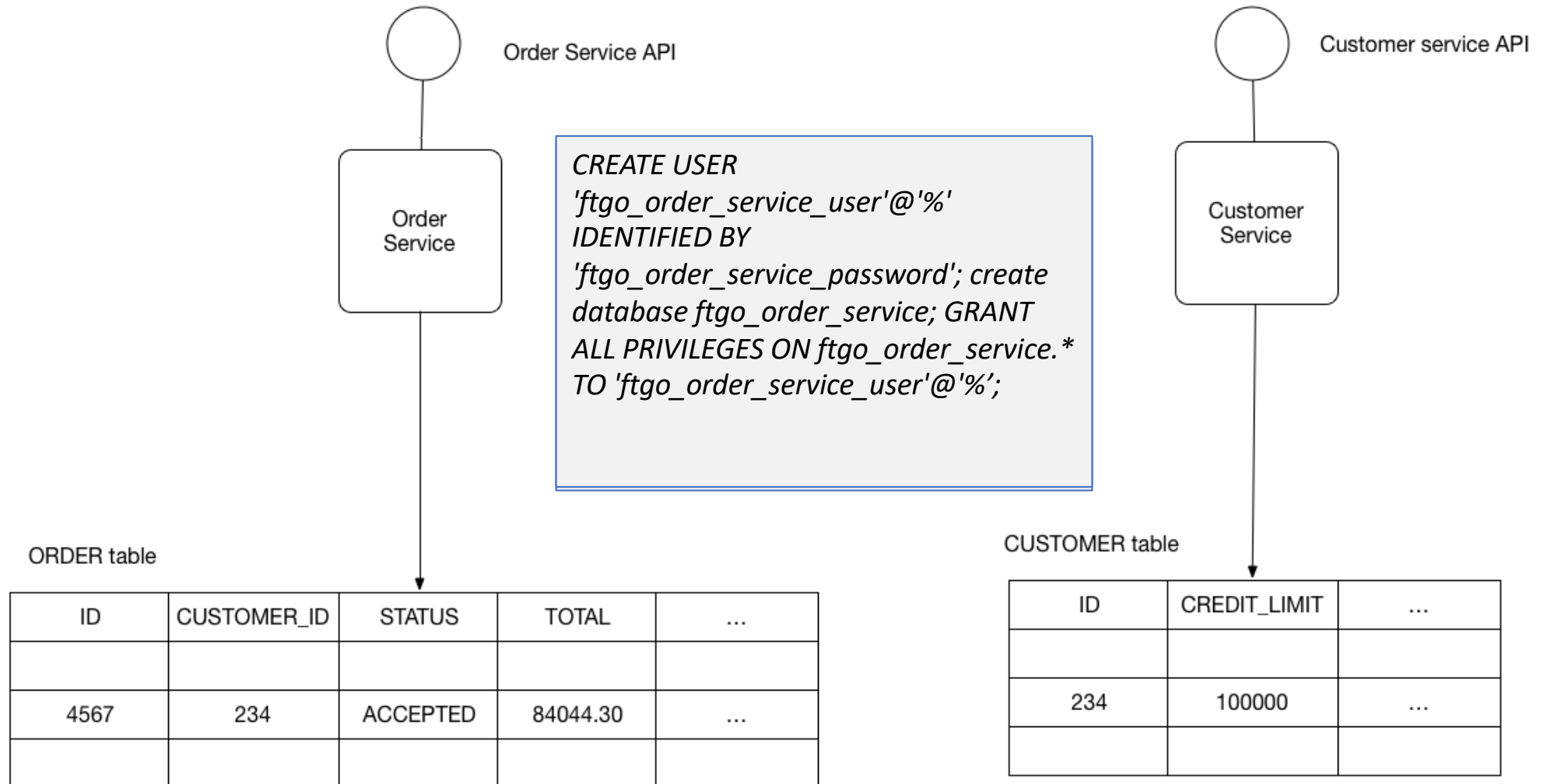


Patrón: Base de Datos por Microservicio

- A fin de incrementar la **autonomía de cada servicio** (por ej., cuando se despliega), cada microservicio funciona con una base de datos dedicada
 - Puede ser una base de datos físicamente separada, lo cual permite distintos tipos de bases de datos (por ej., relacionales, NoSQL, etc.)
 - Pueden ser tablas distintas, y separadas lógicamente, dentro de la misma base de datos
- Otros componentes no conocen dicha base de datos
- Todas las llamadas a otros microservicios se hacen via APIs REST



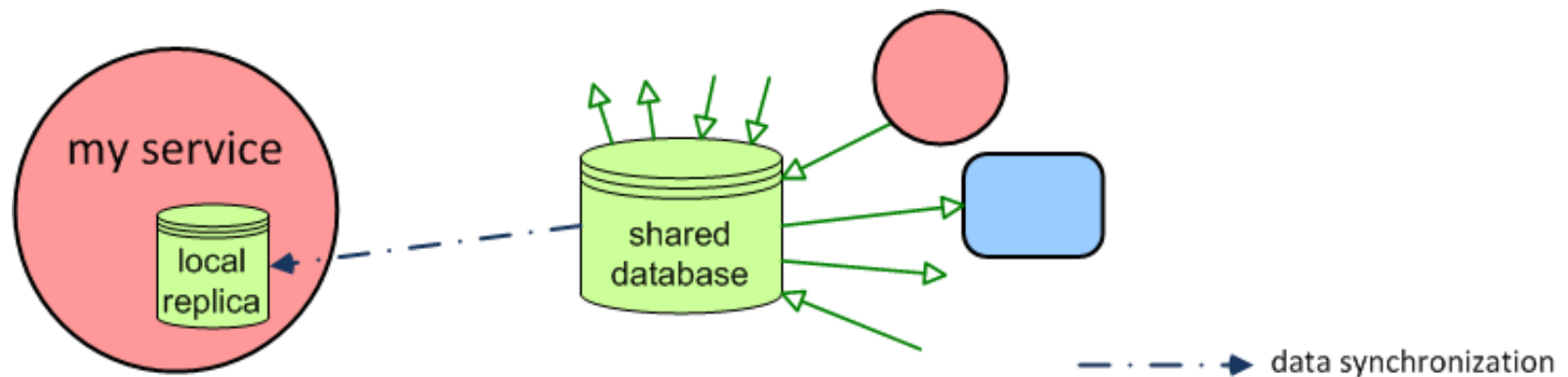
Ejemplo de BD por Microservicio



- Puede requerir sincronizaciones de datos entre las bases de datos

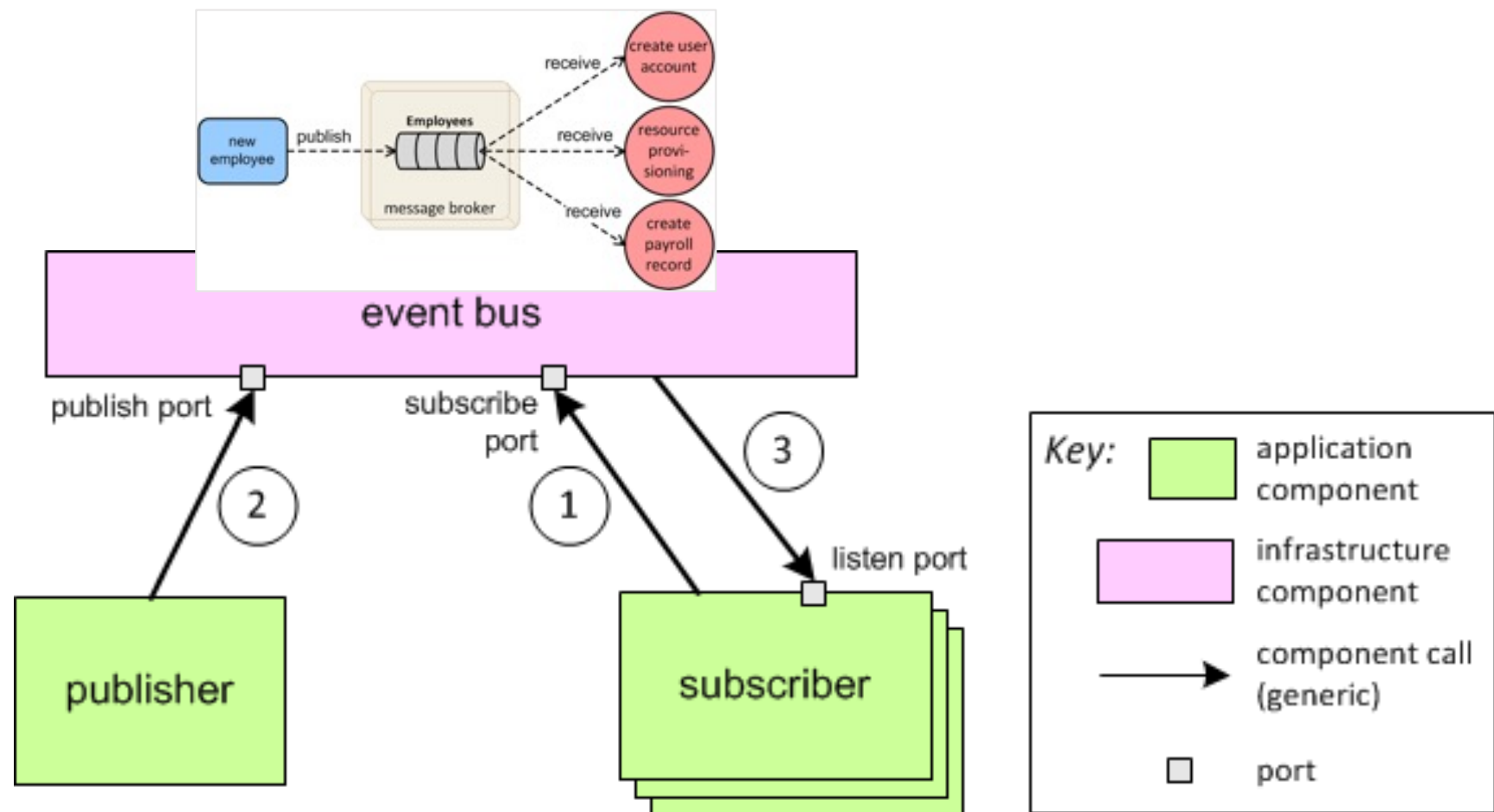
Patrón: Service Data Replication

- Un servicio necesita acceder a datos que pertenecen a otro servicio, o son compartidos con otros
- Una solución es crear una base dedicada para el servicio **que replique los datos requeridos** desde la base maestra
 - Esto requiere un mecanismo de sincronización para actualizar la réplica
 - Triggers, integración vía ETLs, mensajería punto-a-punto a pub/sub
 - Puede generar stale data, o necesidad de sincronizar de ambos lados



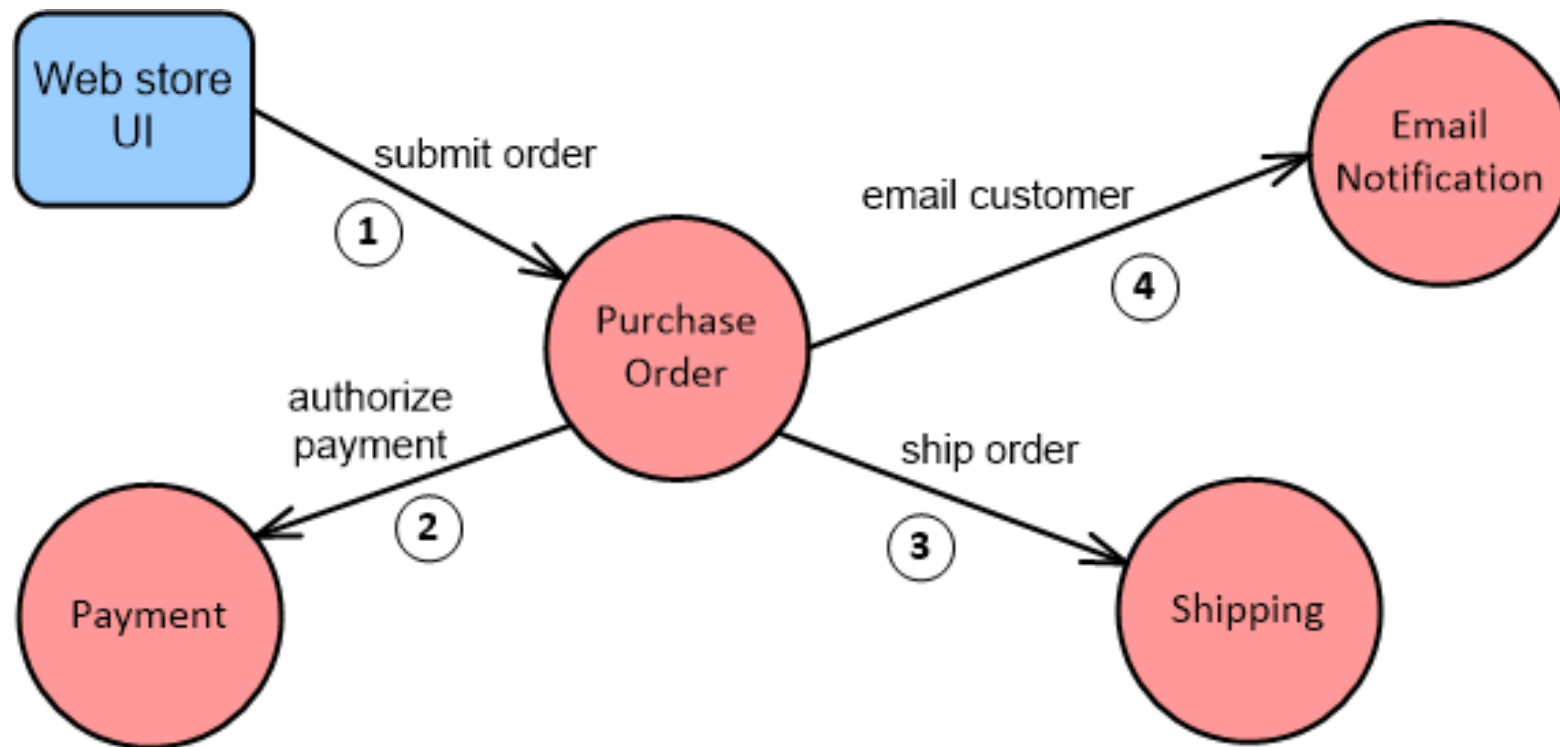
Event-driven Architecture (EDA)

- Es un estilo arquitectónico en el cual los componentes se comunican principalmente a través del envío de mensajes o eventos de manera asincrónica



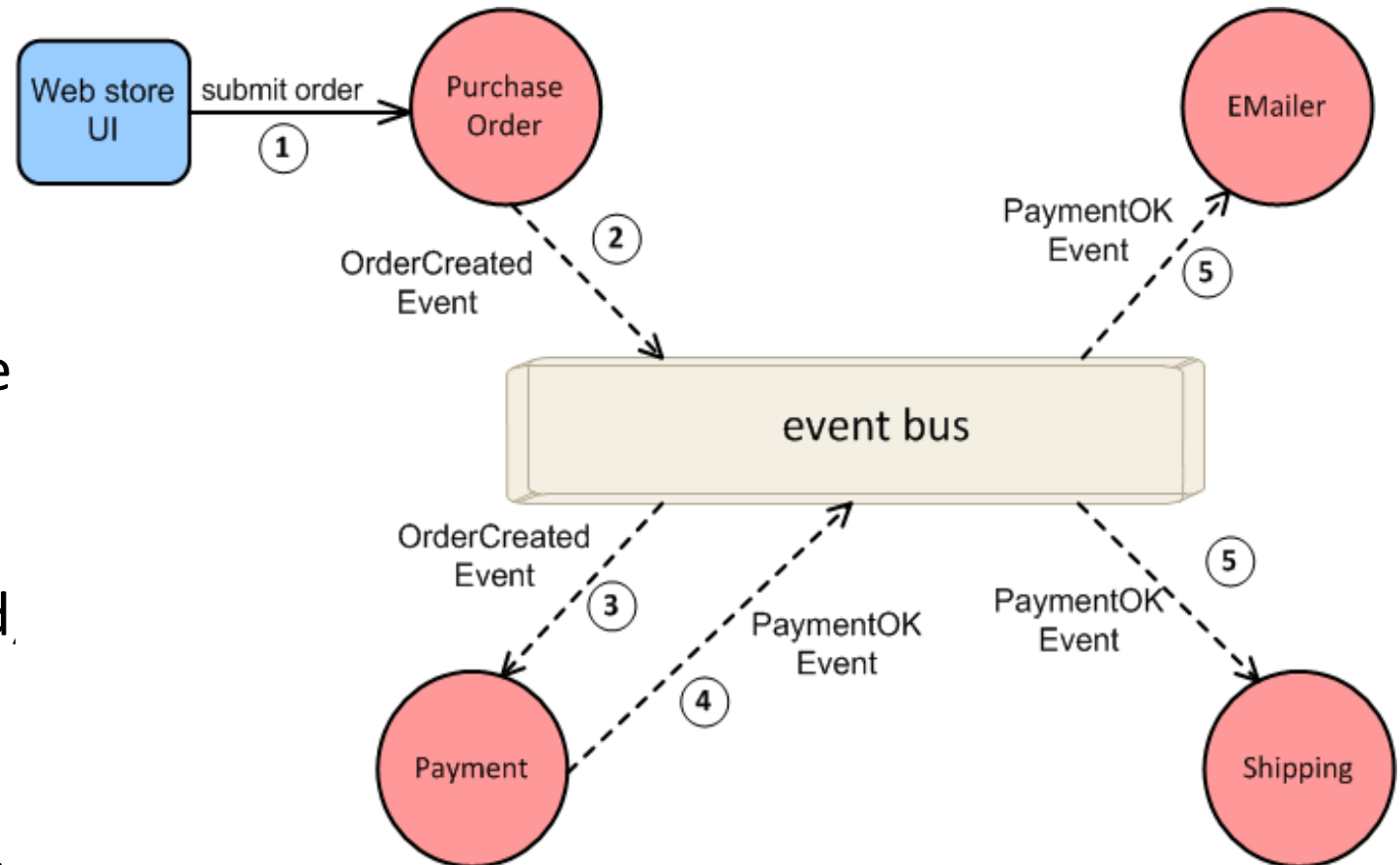
Microservicios sin EDA

- Los consumidores invocan operaciones sobre los servicios
- Las llamadas a los servicios son sincrónicas (solicitud-respuesta)



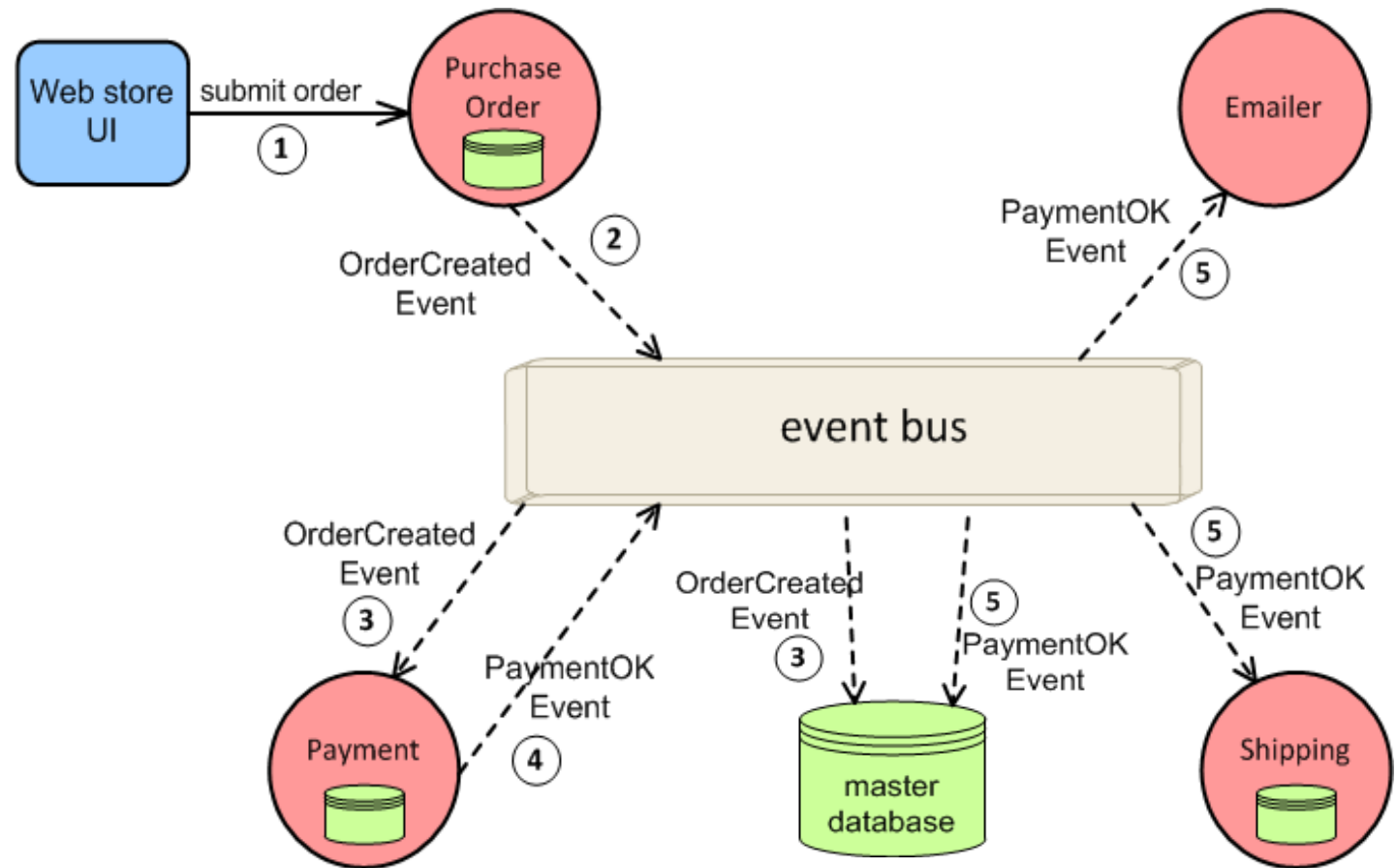
Microservicios con EDA

- En EDA, los publicadores anuncian eventos, y los servicios actúan en base a ellos
- Esto aporta mantenibilidad, escalabilidad y confiabilidad (pero también, complejidad)



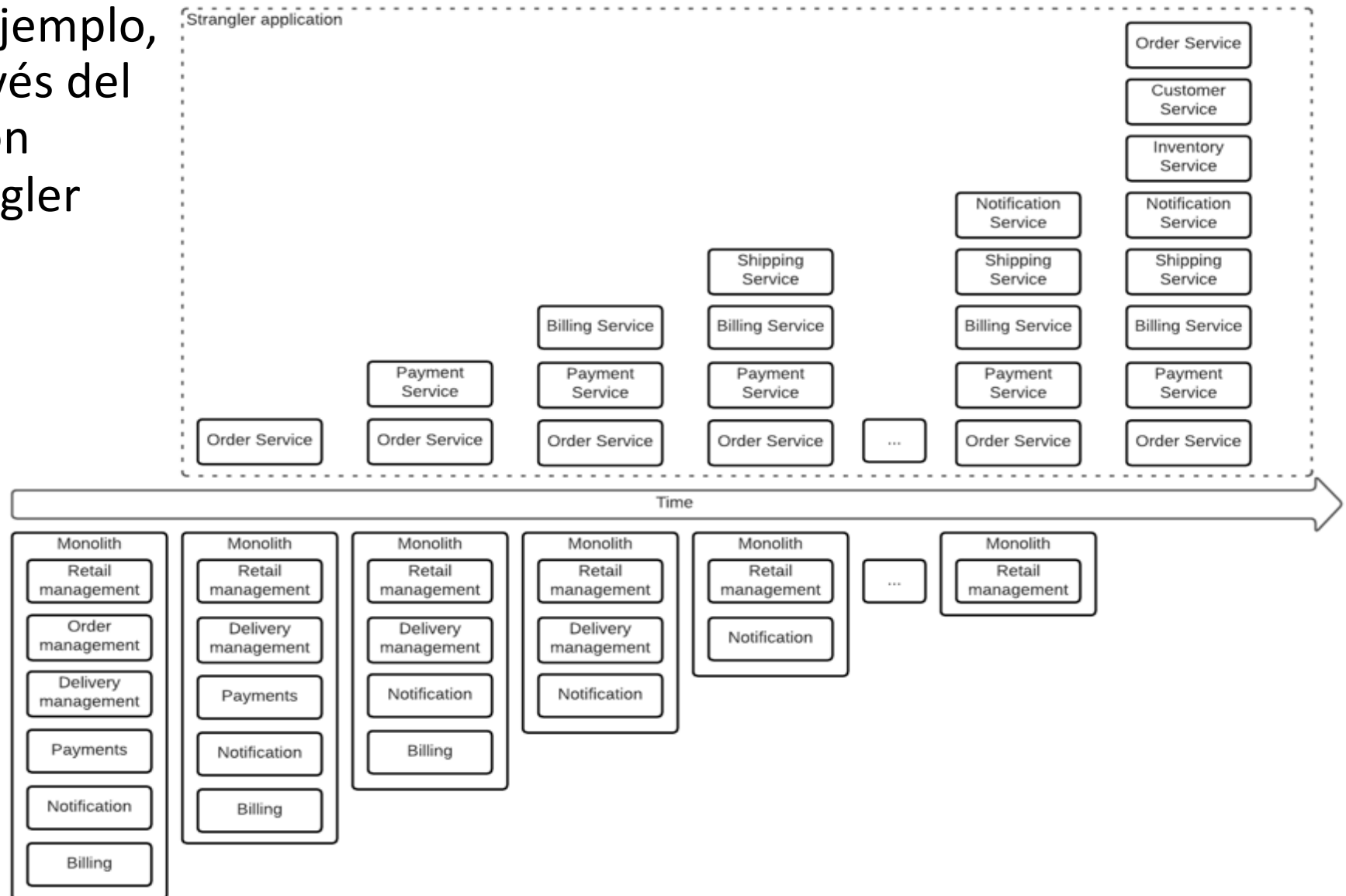
Microservicios con EDA

- Para mejorar la autonomía, se puede crear una base de datos dedicada por microservicio, con una replica de los datos que necesita tomados de una base de datos (master) compartida



Migración (de Monolito) a Microservicios

- Por ejemplo, a través del patrón Strangler



Productividad versus Complejidad

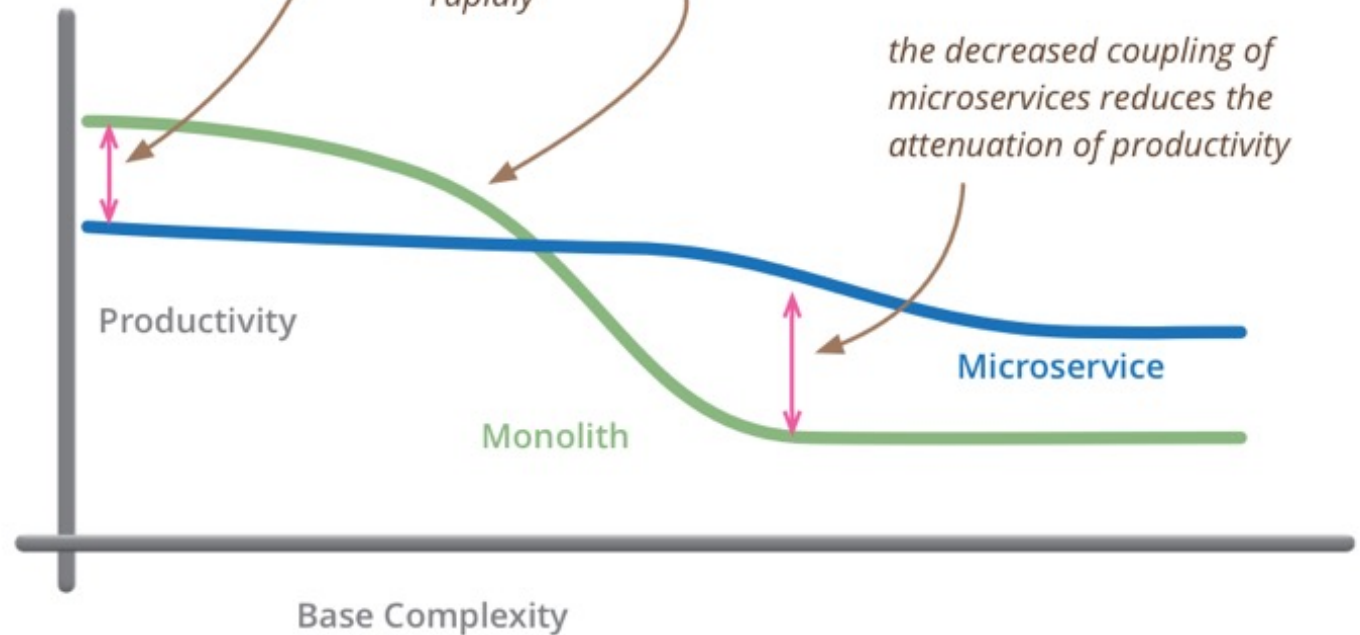
- La complejidad disminuye la productividad
- En sistemas poco complejos, microservicios agrega complejidad técnica que no tiene ROI positivo respecto al monolito
- En sistemas complejos, la independencia de cada equipo ofrece un aumento de productividad que compensa el costo técnico

<https://vuestorefront.io/blog/monolith-vs-microservices>

for less-complex systems, the extra baggage required to manage microservices reduces productivity

as complexity kicks in, productivity starts falling rapidly

the decreased coupling of microservices reduces the attenuation of productivity



but remember the skill of the team will outweigh any monolith/microservice choice

